Лекало

Цифровой дизайнер презентаций — документация

ЛЦТ 2026 · задача №4 VK Tech · команда RealityFlex
Репозиторий: github.com/RealityFlex/LCT_pres

1. Развёртывание и использование README.md
2. Архитектура docs/ARCHITECTURE.md
3. Модели docs/MODELS.md
4. Аудит docs/AUDIT.md

Лекало — цифровой дизайнер презентаций

Сервис принимает корпоративный шаблон `.pptx` и бриф. Шаблон он разбирает на дизайн-систему: палитру, гарнитуры, шкалу кеглей, сетку, холсты и собственные композиции шаблона. По брифу пишет сценарий и тексты, затем собирает **три варианта** колоды в стиле шаблона:

- **A — классический:** максимально близко к шаблону, сбалансированная плотность;
- **B — дизайнерский:** модель компонует каждый содержательный слайд по дизайн-системе шаблона (свои схемы, сетки, акценты) и проверяет себя по скриншоту, код проверяет каждую попытку;
- **C — компактный:** плотнее, таблицы и сравнения, меньше слайдов.

Каждый слайд проходит встроенный аудит: 26 детерминированных проверок и 11 контекстных через VLM. Пользователь сам выбирает, какие замечания исправить. Экспорт — нативный PPTX (тексты, таблицы и диаграммы редактируются), PDF и HTML.

Время на три колоды	обычный режим — не дольше 5 минут (лимит T3): у каждого этапа свой лимит. На инференсе VK — 3:20–4:36 на шаблонах VK, на Yandex — 1,5–2,5 мин. Extended thinking — без лимита, с автоисправлением оформления
Объём	10–15 слайдов в каждом варианте или число, заданное пользователем. Длительность выступления задаёт объём и текст спикера к каждому слайду
Входные данные	бриф + приложенные материалы (txt, md, pdf, docx, pptx): README, документация, отчёты, старая презентация
Шаблоны	любые `.pptx` / `.potx`, в том числе незнакомые и растровые (слайды-картинки из Figma/Canva)
Модели	Qwen — текст и VLM: основной контур — инференс VK (Qwen 27B, подключается переменными `LLM_*`), запасной — Yandex Cloud AI Studio (Qwen3.6-35B-A3B, gpt-oss-20b, Apache 2.0); Kandinsky через GigaChat — иллюстрации
Интерфейс	браузер на десктопе (Chrome, Firefox, Safari, Яндекс Браузер) и мини-приложение ВКонтакте

Документ состоит из двух частей.

Часть I — как развернуть и пользоваться

1. Что понадобится
2. Развёртывание
3. Первый запуск и проверка
4. Как пользоваться: веб-интерфейс
5. Командная строка
6. HTTP API
7. Настройка
8. Что остаётся на диске и воспроизводимость
9. Тесты
10. Если что-то не работает
11. Ограничения

Часть II — как всё устроено

- Архитектура: слои, данные между ними, вёрстка, оркестрация, фронтенд, расширение.
- Модели: какие модели и где используются.
- Аудит: каталог проверок, исправления, защита от ложных срабатываний.
- Карта репозитория — в конце документа.

Часть I. Развёртывание и использование

1. Что понадобится

Что	Зачем	Обязательно
Docker с Compose **или** Python 3.12 + Node 20 + LibreOffice	запуск сервиса	да
`LLM_API_KEY`, `LLM_FOLDER_ID`	доступ к Qwen3.6-35B-A3B и gpt-oss-20b (Yandex Cloud AI Studio, OpenAI-совместимый API)	**да**, без них генерация не запустится
`GIGACHAT_AUTH_KEY`	иллюстрации (Kandinsky через GigaChat)	нет, без ключа колоды собираются без картинок
Шаблон `.pptx` или `.potx`	дизайн-система для колоды	да, загружается из интерфейса

Ресурсы. Своего инференса сервис не запускает: модели вызываются по API, GPU не нужен. На диске понадобится место под профили шаблонов, рендеры слайдов и кэш картинок: порядка десятков мегабайт на прогон.

Куда смотреть в браузере. Интерфейс рассчитан на десктоп. Веб-интерфейс проверяется в актуальной и предыдущей версиях Chrome, Firefox, Safari и Яндекс Браузера на macOS и Windows.

Как получить ключи.

- *Yandex Cloud AI Studio:* создайте сервисный аккаунт и API-ключ в консоли Yandex Cloud, `LLM_FOLDER_ID` — идентификатор каталога, в котором включён доступ к моделям. Имена моделей в `config.yaml` подставляют его вместо `{folder}`.
- *GigaChat:* developers.sber.ru/studio. Нужен «ключ авторизации» (base64 от `client_id:client_secret`) и область доступа `GIGACHAT_API_PERS` для личного тарифа.

2. Развёртывание

2.1. Docker (рекомендуется)

```
git clone https://github.com/RealityFlex/LCT_pres lekalo && cd lekalo
cp .env.example .env         # впишите LLM_API_KEY, LLM_FOLDER_ID, при желании GIGACHAT_AUTH_KEY
docker compose up --build    # интерфейс: http://localhost:8000
```

Что происходит при сборке (`Dockerfile`):

1. Первый этап на Node 20 собирает фронтенд в `frontend/dist`.
2. Второй этап на Python 3.12 ставит LibreOffice (рендер PPTX→PDF), `fontconfig` и базовые кириллические шрифты, затем зависимости из `backend/requirements.txt`.
3. В образ копируются `config.yaml`, `examples/`, `backend/` и собранный фронтенд.
4. Контейнер слушает порт 8000, отвечает на `GET /api/health` (проверка здоровья каждые 30 с) и перезапускается сам (`restart: unless-stopped`).

Каталог `./data` монтируется в контейнер как `/app/data`: профили шаблонов, проекты, кэш картинок и шрифтов переживают перезапуск.

Остановить: `docker compose down`. Обновить после `git pull`: `docker compose up --build -d`.

2.2. Локально (Windows / Linux / macOS)

Нужны Python 3.12, Node 20 и LibreOffice (команда `soffice`).

```
pip install -r backend/requirements.txt
npm --prefix frontend ci
npm --prefix frontend run build
cp .env.example .env         # впишите ключи
python -m uvicorn app.api.main:app --app-dir backend --port 8000
```

Интерфейс откроется на http://localhost:8000: FastAPI раздаёт собранный фронтенд и API с одного порта.

Рекомендуется виртуальное окружение (`python -m venv .venv`). Версии зависимостей закреплены в `backend/requirements.txt` — это те версии, на которых проверен пайплайн.

Где искать LibreOffice. Сначала `SOFFICE_PATH` из `.env`, затем `render.soffice_path` из `config.yaml`, затем `soffice` и `libreoffice` в `PATH`, затем стандартные каталоги установки:

- Windows: `C:\Program Files\LibreOffice\program\soffice.exe`;
- Linux: `/usr/bin/soffice`;
- macOS: `/Applications/LibreOffice.app/Contents/MacOS/soffice`.

Если LibreOffice установлен в другое место, задайте `SOFFICE_PATH` в `.env`.

2.3. Режим разработки фронтенда

```
python -m uvicorn app.api.main:app --app-dir backend --port 8000   # бэкенд
npm --prefix frontend run dev                                       # http://localhost:5173
```

Vite проксирует `/api` на `127.0.0.1:8000`. Проверка типов: `npm --prefix frontend run typecheck`.

2.4. Переменные окружения

Секреты хранятся только в `.env` (он в `.gitignore`). Шаблон — `.env.example`.

Переменная	Назначение
`LLM_API_KEY`	ключ API Yandex Cloud AI Studio. Обязательна
`LLM_FOLDER_ID`	идентификатор каталога Yandex Cloud, подставляется в имена моделей. Обязательна для Yandex Cloud
`LLM_BASE_URL`	необязательно: переопределяет `llm.base_url` из `config.yaml` (свой vLLM или другой OpenAI-совместимый эндпоинт)
`LLM_PROJECT_ID`	необязательно: заголовок `Project-ID` для эндпоинтов, которые его требуют
`LLM_TEXT_MODEL`, `LLM_VISION_MODEL`, `LLM_FALLBACK_MODEL`	необязательно: имена моделей вместо заданных в `config.yaml`
`GIGACHAT_AUTH_KEY`	ключ авторизации GigaChat для иллюстраций
`GIGACHAT_SCOPE`	область GigaChat, по умолчанию `GIGACHAT_API_PERS`
`DATA_DIR`	каталог данных, по умолчанию `./data` (в Docker — `/app/data`)
`SOFFICE_PATH`	путь к `soffice`
`APP_CONFIG`	путь к другому файлу конфигурации вместо `config.yaml`

2.5. Замечания по эксплуатации

- **Нет аутентификации.** API открыт, CORS разрешён для всех источников. Для доступа из сети ставьте сервис за обратный прокси с авторизацией и не выставляйте порт 8000 в открытый интернет.
- **Прогресс идёт через SSE** (`/api/jobs/{id}/events`). Если сервис стоит за nginx, отключите буферизацию для этого пути (`proxy_buffering off`), иначе индикатор прогресса будет обновляться рывками.
- **Состояние задач хранится в памяти процесса и на диске.** Если сервер перезапущен посреди генерации, задача помечается ошибкой и запускается заново из интерфейса.
- **Один процесс uvicorn.** Задачи живут в памяти процесса, поэтому не запускайте несколько воркеров на одном каталоге `data`.

3. Первый запуск и проверка

1. Проверьте, что сервис видит модели и LibreOffice:

   ```
   curl http://localhost:8000/api/health
   ```

   Ответ содержит:

   - `llm.configured` — `true`, если задан `LLM_API_KEY`;
   - `llm.model` и `llm.vision` — используемые модели;
   - `images.enabled` — `true`, если есть ключ GigaChat;
   - `soffice` — путь к LibreOffice или текст ошибки;
   - `variants` и `deadline_s` — активные варианты вёрстки и общий дедлайн прогона.

2. Откройте http://localhost:8000, перейдите в «Шаблоны» и загрузите любой `.pptx`. Разбор занимает от 20 секунд до двух минут в зависимости от числа слайдов и шрифтов (на разобранных шаблонах — 20–107 с). Повторная загрузка того же файла мгновенна: профиль кэшируется по sha256.
3. Нажмите «Новая презентация», выберите шаблон, нажмите одну из кнопок «Заполнить примером» и запустите генерацию. Через 1–2 минуты появятся три варианта.

4. Как пользоваться: веб-интерфейс

4.1. Шаблоны

Раздел «Шаблоны» — библиотека разобранных шаблонов.

- **Загрузка.** Перетащите `.pptx` или `.potx` в область загрузки. Можно грузить и на шаге 1 мастера новой презентации.
- **Разбор.** Сервис извлекает палитру, гарнитуры, шкалу кеглей, поля, «холсты» (слайды без контента), образцы композиций и нарратив шаблона. Этап и прогресс разбора виден в области загрузки.
- **Просмотр.** В карточке шаблона: палитра, типографика, поля, список холстов с превью, слайды исходника с пометками (служебные слайды, группы карточек), формат повествования.
- **Удаление.** Убирает профиль шаблона; готовые проекты остаются.

Подходят любые шаблоны. Шрифты берутся из файла, недостающие подкачиваются из Google Fonts. Растровые шаблоны (слайды-картинки из Figma или Canva) восстанавливаются приближённо, см. ограничения.

4.2. Новая презентация

Мастер из двух шагов; справа сводка и кнопка запуска.

Шаг 1. Шаблон. Выберите карточку или загрузите новый файл.

Шаг 2. Бриф.

Поле	Что писать	Влияние
О чём презентация	тема, 3–140 знаков. Обязательное поле	заголовок колоды и её сюжет
Назначение	проект / продукт / фича / инициатива / обучение / другое	сюжетная арка: например, для фичи «проблема → как работает → метрики → план запуска → риски»
Аудитория	кто будет смотреть	тон и глубина изложения
Данные и факты	цифры, результаты, сроки, риски	**единственный источник цифр и фактов.** Числа, которых здесь нет, на слайды не попадают. Ряд значений даёт диаграмму, отдельные числа — крупные цифры
Материалы	файлы txt, md, pdf, docx, pptx и другие текстовые (csv, json, yaml, html, rst); до 10 файлов за раз	их текст (до 20 тыс. знаков на файл и 30 тыс. в сумме) становится источником фактов наравне с полем выше
Длительность выступления	не важно / 5 / 7 / 10 / 15 / 20 мин	объём колоды (~1,5 слайда в минуту, в пределах 10–15) и объём текста спикера (130 слов в минуту)

Поле	Что писать	Влияние
Иллюстрации	где уместно / обязательно / без картинок	«обязательно» — 2–3 слайда с иллюстрацией; без ключа GigaChat картинок не будет в любом режиме
Количество слайдов	автоматически / задать (5–20)	«автоматически» — 10–15 по объёму брифа
Автор	имя или команда	попадает на титульный слайд

Черновик брифа сохраняется в браузере (`localStorage`), поэтому не пропадёт при обновлении страницы.

Запуск: кнопка «Сгенерировать 3 варианта», `Enter` в поле темы или `Ctrl` + `Enter`.

Советы, как получить лучший результат.

- Пишите в «Данные и факты» конкретные числа с единицами и периодами: «время сборки 4,5 → 1,5 часа», «бюджет 48 млн руб.». Без цифр презентация будет качественной: карточки, процессы, сравнения.
- Чтобы получить график, дайте ряд значений: «янв 4,5; фев 4,2; мар 3,1…».
- Если тема шире брифа, приложите README, отчёт или старую презентацию.
- Задайте длительность выступления, если презентацию будут читать вслух: вы получите и нужный объём, и текст спикера.

Примеры брифов лежат в `examples/`. Их же предлагает кнопка «Заполнить примером».

4.3. Ход генерации

После запуска открывается экран прогресса. Этапы:

Этап	Что происходит
Сценарий колоды	модель пишет структуру, заголовки и тексты. Числа не из брифа убираются до вёрстки
Иллюстрации	(необязательный) генерация картинок для слайдов-иллюстраций
Вёрстка трёх вариантов	раскладка по сетке и стилю шаблона; тексты, не влезшие в макет, сокращаются
Сборка и рендер	PPTX с нативными объектами и превью слайдов
Аудит	детерминированные проверки и VLM-ревью каждого слайда
Экспорт	PPTX, PDF, HTML

На экране видны найденный сценарий и заготовки вариантов по мере готовности. Прогон держится в пределах 300 с: при нехватке времени контекстный аудит сокращается, колоды не теряются.

4.4. Проект: просмотр, аудит, экспорт

Когда генерация завершена, открывается страница проекта.

- **Вкладки A / B / C.** Три варианта вёрстки одного и того же контента.
- **Лента слайдов и просмотр.** Слева миниатюры, в центре крупный слайд. Навигация: `←` / `→` или `PageUp` / `PageDown`.

- **Заметки спикера.** Кнопка «Заметки» под слайдом показывает текст спикера; он же записывается в заметки PPTX.
- **Сравнить варианты.** Сетка из трёх вариантов рядом, клик открывает слайд.
- **Показ.** Полноэкранный режим: клавиша `F` или кнопка «Показ». Листать `←` / `→`, пробел и `Enter`, выход — `Esc`.
- **Аудит.** См. следующий раздел.
- **Меню «Ещё».**
  - «Тот же контент на другом шаблоне» — сценарий и тексты сохраняются, меняется только дизайн-система. Не тратит время на новый сценарий.
  - «Сгенерировать заново» — новый сценарий по тому же брифу.
  - «Поделиться ссылкой» — ссылка на проект; в мини-приложении ВКонтакте открывает диалог «поделиться».
  - «Удалить проект» — недоступно, пока идёт генерация.
- **Скачивание.** Форматы PPTX, PDF и HTML для каждого варианта отдельно.

Форматы экспорта.

- **PPTX** — нативные объекты: тексты, таблицы и диаграммы редактируются в PowerPoint, макеты берутся из шаблона. Слайды не сохраняются растровой картинкой.
- **PDF** — рендер LibreOffice.
- **HTML** — один файл со встроенным просмотрщиком: векторные SVG-страницы, навигация клавишами `←` / `→`, `F` — показ на весь экран, `Esc` — выход. Текст слайдов доступен экранным дикторам.

4.5. Аудит и исправления

Панель «Аудит слайдов» показывает замечания по слайдам; при наведении зона замечания подсвечивается на превью.

- **Фильтры:** по типу — «Все», «Оформление» (детерминированные проверки), «Смысл» (контекстные, через модель) — и по важности: ошибка, предупреждение, совет.
- **Замечания с пометкой «исправимо»** можно отметить флажком. «Выбрать исправимые» отмечает все сразу.
- **«Исправить выбранные»** применяет исправления, пересобирает вариант и проверяет заново. Детерминированный аудит повторяется целиком, контекстный — только по изменённым слайдам. Каждое исправление повышает номер ревизии.
- Часть замечаний исправить автоматически нельзя (например, слабая логика перехода). Они показываются без флажка.

Каталог всех 37 проверок и способов исправления — в docs/AUDIT.md.

4.6. Данные: таблицы → выводы и диаграммы

В поле «Материалы» можно приложить таблицы: CSV/TSV, XLSX/XLS, табличный JSON, а также текст (md, txt, docx) с таблицами внутри. Каждый файл получает карточку с профилем: строки, измерения и метрики, временная ось, пропуски, предлагаемая задача. В карточке можно написать цель («показать связь цены и продаж») — она меняет выбор задачи и диаграммы.

Что делает сервис с каждым датасетом:

1. **Профиль:** тип и роль колонок, пропуски, выбросы (IQR), иерархии, частота дат. Широкие таблицы с периодами в заголовках разворачиваются.
2. **Задача:** динамика, сравнение, структура долей, распределение, взаимосвязь, справочные значения. Выбор идёт по профилю и цели — из карточки или из предложения брифа, которое говорит об этом датасете. Формат файла на выбор не влияет.
3. **Визуализация:** 2–3 варианта с баллами и объяснением (линия, столбцы, полосы, сгруппированные и накопительные, кольцо/круг, гистограмма, рассеяние, таблица, карточки KPI). Один и тот же датасет на разных шаблонах получает один и тот же тип.
4. **Слайд:** заголовок-вывод с числом, подписи осей с единицами, источник в подвале, блок «почему это важно / что из этого следует», ключевые цифры сбоку.
5. **Колода:** титул → «Главное» → план → слайды данных → выводы → что делать → финал. Методика (профиль каждого файла, выбранная визуализация и почему) — в заметках спикера к последнему слайду с данными.

Сложные случаи: больше 12 категорий — два слайда (топ и остальные) или «Прочие»; длинные ряды агрегируются; файл без чисел в презентацию не попадает — в форме он помечен «не распознано» с причиной.

На странице проекта у слайда с диаграммой есть панель «Визуализация»: почему выбран тип, альтернативы и кнопка «Показать так» — слайд пересобирается за несколько секунд.

Примеры: `examples/data_demo` (сеть кофеен, 7 датасетов), `examples/data_school` (онлайн-школа: ряд по неделям, анкета с пропусками, md с широкой таблицей), `examples/data_carsharing` (365 дней «погода → спрос», 25 районов, файл без чисел). Генераторы — `make_datasets.py` в каждой папке.

4.7. Брендбук

Блок «Брендбук (необязательно)» на шаге оформления. Форматы:

- `.fig` — файл Figma (Файл → Save local copy): цвета из стилей и заливок, шрифты по доле использования, логотипы (векторы → PNG);
- JSON дизайн-токенов (W3C `$value`, Tokens Studio, плоский `{"colors":…, "fonts":…}`), включая отступы и правила логотипа;
- PDF-гайдлайн: HEX/RGB/CMYK с подписями, шрифты, минимальный размер и охранное поле логотипа;
- PNG/SVG/JPG или ZIP с логотипами и примерами слайдов.

Карточка показывает палитру по ролям, шрифты, логотипы, правила и предупреждения. Приоритет по умолчанию у брендбука: для встроенного стиля подменяются цвета, шрифты, логотип и сетка; для шаблона компании собирается производный шаблон (тема, явные цвета на слайдах, шрифты, замена логотипа). Переключатель «шаблон важнее» оставляет от брендбука только логотип. Без брендбука всё работает на цветах шаблона.

Пример в четырёх форматах: `examples/brand_zerno/` (`tokens.json`, `guideline.pdf`, `logos/`, `slides/`, `brand_zerno.zip`).

5. Командная строка

Командная строка запускает тот же пайплайн без веб-интерфейса. Так удобно проверять воспроизводимость и гонять пакеты.

```
cd backend

# разбор шаблона → data/templates/<id>/profile.json
python -m app.cli parse ../path/template.pptx

# полный прогон: бриф → три варианта
python -m app.cli generate --template ../path/template.pptx --brief ../examples/brief_feature.yaml

# по id уже разобранного шаблона
python -m app.cli generate --template <id> --brief ../examples/brief_sklad.yaml

# без планировщика: готовая контент-модель
python -m app.cli generate --template <id> --content tests/fixtures/demo_content.json

# без ключей: разбор без VLM
python -m app.cli parse ../path/template.pptx --no-llm
```

Параметры:

Команда	Параметр	Смысл
`parse`	`template`	путь к `.pptx`
	`--force`	разобрать заново, игнорируя кэш
	`--no-llm`	без VLM-описания стиля (профиль остаётся полным)
`generate`	`--template`	путь к `.pptx` или id разобранного шаблона
	`--brief`	YAML-файл брифа
	`--topic`, `--purpose`, `--audience`, `--details`, `--slides`	бриф прямо в командной строке вместо файла
	`--content`	готовая контент-модель JSON: планирование пропускается
	`--no-llm`	не использовать VLM при разборе шаблона

Формат брифа (YAML, поля модели `Brief`):

```
topic: Умные напоминания в мобильном банке   # обязательное
purpose: фича                                # фича | продукт | проект | инициатива | обучение | другое
audience: продуктовая команда
details: |                                   # единственный источник цифр
  A/B-тест на 120 тыс. пользователей, 6 недель.
  Просрочки снизились с 7,8% до 5,1%.
slide_count: null                            # число слайдов или null — автоматически (10–15)
duration_min: 10                             # длительность выступления, мин
author: ""
language: ru
images: auto                                 # auto | on | off
```

Поля `title_hint` в примерах — подпись кнопки «Заполнить примером» в интерфейсе, на генерацию они не влияют.

Каждый прогон пишет результат в `data/jobs/<id>/` (см. раздел 8), а в консоль печатает этапы со временем и итоговую сводку: варианты, число слайдов, число замечаний, использованные версии скиллов, статистику вызовов модели.

6. HTTP API

Интерфейс общается с сервисом по HTTP; те же вызовы доступны любому клиенту. Интерактивная документация FastAPI — `http://localhost:8000/docs`.

Метод и путь	Что делает
`GET /api/health`	состояние: ключи, модели, LibreOffice, варианты
`GET /api/stats`	число шаблонов и проектов, среднее время прогона
`GET /api/templates`	список разобранных шаблонов
`POST /api/templates`	загрузка `.pptx` / `.potx` (`multipart`, поля `file` и необязательное `force`). Ответ: `{id, status}`, где статус `ready` или `parsing`
`GET /api/templates/{id}/status`	прогресс разбора: `parsing` / `ready` / `error`
`GET /api/templates/{id}`	полный профиль: палитра, типографика, холсты, слайды
`DELETE /api/templates/{id}`	удалить шаблон
`GET /api/templates/{id}/slide/{n}.png`	превью слайда шаблона; `?w=` — миниатюра WebP
`GET /api/templates/{id}/canvas/{cid}.png`	превью холста
`POST /api/materials`	текст и датасеты из файлов (`multipart`, поле `files`): для данных — `dataset {id, tables[]}` с профилем, задачей и вариантами визуализации

Метод и путь	Что делает
`POST /api/brand`	брендбук (`multipart`, поле `files`: `.fig`, `.json`, `.pdf`, `.png` / `.svg`, `.zip`). Ответ: палитра, шрифты, логотипы, сетка, правила, предупреждения
`GET /api/brand/{id}` · `GET /api/brand/{id}/logo/{i}.png`	карточка брендбука и его логотипы
`POST /api/jobs/{id}/variants/{vid}/slides/{n}/chart`	другая визуализация слайда данных из предложенных: `{type}`
`POST /api/jobs`	запуск генерации: `{template_id, brief, materials[]}`. Ответ: `{id}`
`GET /api/jobs`	список проектов
`GET /api/jobs/{id}`	состояние задачи: этапы, варианты, статистика
`GET /api/jobs/{id}/events`	поток прогресса (SSE)
`GET /api/jobs/{id}/variants/{A|B|C}`	слайды варианта и отчёт аудита
`GET /api/jobs/{id}/variants/{vid}/slide/{n}.png`	превью слайда; `?w=` — миниатюра
`POST /api/jobs/{id}/variants/{vid}/fix`	применить исправления: `{issue_ids: [...]}`
`GET /api/jobs/{id}/variants/{vid}/download/{pptx|pdf|html}`	скачать файл; `?inline=true` — открыть в браузере
`GET /api/jobs/{id}/content`	контент-модель (сценарий)
`POST /api/jobs/{id}/rerun`	тот же бриф на другом шаблоне: `{template_id, reuse_content}`
`DELETE /api/jobs/{id}`	удалить проект (409, пока идёт генерация)
`GET /api/examples`	примеры брифов из `examples/`
`GET /api/skills`	скиллы: версии, активная версия, описание
`GET /api/audit/checks`	каталог проверок аудита

Пример: от шаблона до файла.

```
# 1. загрузить шаблон
curl -F file=@template.pptx http://localhost:8000/api/templates
# {"id":"156543731f0b53b9","status":"parsing"}

# 2. дождаться разбора
curl http://localhost:8000/api/templates/156543731f0b53b9/status

# 3. запустить генерацию
curl -X POST http://localhost:8000/api/jobs -H "Content-Type: application/json" -d '{
  "template_id": "156543731f0b53b9",
  "brief": {"topic": "Умные напоминания в мобильном банке", "purpose": "фича",
            "details": "Просрочки снизились с 7,8% до 5,1%."}
}'
# {"id":"20260929-101500-a1b2c3"}

# 4. следить за прогрессом (SSE) или опрашивать состояние
curl -N http://localhost:8000/api/jobs/20260929-101500-a1b2c3/events
curl http://localhost:8000/api/jobs/20260929-101500-a1b2c3

# 5. забрать результат
curl -OJ "http://localhost:8000/api/jobs/20260929-101500-a1b2c3/variants/A/download/pptx"
```

7. Настройка

Всё, что влияет на результат, живёт в `config.yaml`. Изменения подхватываются при перезапуске сервиса.

Раздел	Что настраивается
`llm`	провайдер, `base_url`, модели `text_model` / `vision_model` / `fallback_model` (в именах `{folder}` заменяется на `LLM_FOLDER_ID`), `temperature`, `timeout_s`, `max_concurrency`, `disable_thinking`
`images`	`provider` (`gigachat` или `none`), `model`, `max_per_deck` (максимум иллюстраций на колоду), `timeout_s`
`render`	`soffice_path`, `max_parallel` (одновременных рендеров), `preview_dpi`
`pipeline`	`default_slide_count`, `min_slides`, `max_slides`, `deadline_s` (общий дедлайн прогона, 300 с), `contextual_audit` (`false` отключает VLM-аудит)
`variants`	описание трёх вариантов: `id`, `name`, `strategy` (`faithful` / `visual` / `dense`), `description`
`audit`	пороги проверок: 6 буллетов, 15 слов в пункте, таблица 7×5, 5 серий, заполненность ¼–¾, контраст 4,5:1, две гарнитуры
`skills`	активная версия каждого скилла (промпта): файлы `backend/skills/<имя>/<версия>.yaml`

Типичные сценарии.

- *Свой инференс вместо облака:* укажите `llm.base_url` (или `LLM_BASE_URL`) и имена моделей своего OpenAI-совместимого сервера (vLLM, инференс VK). Код менять не нужно.
- *Без иллюстраций:* `images.provider: none` или не задавайте `GIGACHAT_AUTH_KEY`.

- *Быстрее и дешевле:* `pipeline.contextual_audit: false` пропускает VLM-проверки слайдов, детерминированный аудит остаётся.
- *Переключить версию промпта:* смените `skills.<имя>` на другую версию из `backend/skills/<имя>/`. Промпты не правятся на месте: изменение — это новый файл версии.

8. Что остаётся на диске и воспроизводимость

Каталог данных (`data/`, в Docker — `/app/data`) целиком создаётся сервисом:

```
data/
  templates/<sha256[:16]>/     разобранные шаблоны
    template.pptx              исходный файл
    profile.json               дизайн-система: палитра, типографика, холсты, паттерны
    preview/, canvases/        превью слайдов и холстов
  jobs/<дата-время-id>/        один проект = один прогон
    job.json                   состояние: бриф, этапы, варианты, версии скиллов, статистика
    content.json               сценарий (контент-модель)
    llm_trace.jsonl            все вызовы моделей: скилл@версия, промпты, ответ, время, токены
    A/ B/ C/                   по варианту:
      plan.json                какой рецепт и холст применены к каждому слайду
      layouts.json             промежуточное представление вёрстки
      audit.json               отчёт аудита
      deck_<вариант>.pptx / .pdf / <вариант>.html
      slides/slide-NNN.png     превью слайдов
  cache/                       кэш картинок, шрифтов, временных файлов
```

Что воспроизводимо, а что нет.

- Разбор шаблона, вёрстка, сборка и детерминированный аудит не используют модели и дают одинаковый результат на одинаковых входных данных.
- Текст сценария и контекстный аудит зависят от модели и на повторных запусках могут различаться.
- Чтобы повторить сборку точно, подайте сохранённый `content.json`: `python -m app.cli generate --template <id> --content data/jobs/<id>/content.json`. Сценарий пропускается, вёрстка и аудит выполняются заново.
- Версии зависимостей закреплены в `backend/requirements.txt`, конфигурация — в `config.yaml`, промпты — в версионируемых файлах скиллов. В `job.json` и `llm_trace.jsonl` каждого прогона записаны имя, версия и хэш использованного промпта.

9. Тесты

```
cd backend && python -m pytest tests -q
```

Текущий результат: 33 теста проходят, 1 пропущен (проверка реальных шаблонов из `data/templates`, если они не загружены).

Модульные тесты (`tests/test_units.py`):

- цвета и WCAG-контраст;
- запись текста с сохранением форматирования шаблона;
- подбор кегля;
- распознавание групп карточек (с подложками и без);
- аппроксимация фона растровых шаблонов;
- фильтры контекстного аудита (ложные «опечатки», стилистические правки);
- сверка чисел сценария с брифом;
- объём колоды по длительности, добор и обрезка слайдов;
- извлечение текста из приложенных материалов;
- восстановление зависших задач.

Интеграционные тесты (`tests/test_pipeline.py`): синтетический шаблон → профиль → три варианта на фиксированном плане → рендер LibreOffice → проверка, что файлы открываются, варианты различаются, объекты нативные, а детерминированный аудит не находит ошибок вёрстки. Интеграционные тесты пропускаются без LibreOffice.

Список проверок аудита и их покрытие — в docs/AUDIT.md.

10. Если что-то не работает

Симптом	Причина и решение
`LLM_API_KEY не задан (.env)`	нет `.env` или ключа. Скопируйте `.env.example` в `.env` и заполните. В Docker после правки `.env` перезапустите контейнер
`модель недоступна: …`	ключ неверный, нет доступа к модели в каталоге, сеть или лимиты. Проверьте `LLM_FOLDER_ID` и включён ли доступ к моделям в Yandex Cloud. Сервис сам пробует запасную модель
`LibreOffice (soffice) не найден. Укажите SOFFICE_PATH.`	установите LibreOffice или задайте `SOFFICE_PATH` в `.env`. В `GET /api/health` поле `soffice` покажет, что найдено
Нет иллюстраций	не задан `GIGACHAT_AUTH_KEY`, либо `images.provider: none`, либо в брифе «Без картинок». `GET /api/health` → `images.enabled`
Ошибка GigaChat OAuth	ключ авторизации неверный или не совпадает область (`GIGACHAT_SCOPE`)
Иллюстрации идут медленно	личный тариф GigaChat не принимает параллельные запросы: одна картинка ≈ 20–50 с
`Нужен файл .pptx или .potx`	загрузка принимает только PowerPoint. `.ppt`, Keynote и Google Slides сначала сохраните как `.pptx`
`В шаблоне не найдено ни одного слайда, пригодного как холст`	шаблон пустой: в нём нет слайдов с заголовком и фоном. Загрузите шаблон с готовыми примерами слайдов
Шаблон разобран, но шрифты подменились	у шрифта нет кириллицы или он не найден: текст меряется запасным шрифтом рендерера. Положите шрифты в систему или используйте шаблон со встроенными шрифтами

Симптом	Причина и решение
Файл материалов не прочитан	`в файле нет текста (скан или пустой документ)` — сканы без текстового слоя не поддерживаются; формат вне txt, md, pdf, docx, pptx и других текстовых не читается
`Генерация прервана: сервер был перезапущен`	сервер перезапускали во время прогона. Запустите проект заново («Сгенерировать заново»)
`генерация ещё идёт` (409) при удалении	дождитесь конца прогона
Прогресс обновляется рывками за прокси	отключите буферизацию SSE (`proxy_buffering off` в nginx)
Генерация упирается в 5 минут	уменьшите `llm.max_concurrency` при лимитах провайдера, отключите `pipeline.contextual_audit`, задайте `images.provider: none`
`npm ci` падает при сборке в Docker	так и задумано: lock-файл собран на Windows, поэтому `Dockerfile` разрешает зависимости заново по диапазонам `package.json`
Порт 8000 занят	смените порт в `docker-compose.yml` (`"8080:8000"`) или в аргументе `--port`

Подробный журнал: `docker compose logs -f lekalo` или вывод `uvicorn`. Полный обмен с моделями по конкретному проекту — `data/jobs/<id>/llm_trace.jsonl`.

11. Ограничения

- **Проверено в LibreOffice, но не в PowerPoint.** Метрики текста считаются по реальным шрифтам шаблона. Если у шрифта нет кириллицы, измерение идёт по запасному шрифту рендерера, и в PowerPoint отрисовка может немного отличаться.
- **Цифры и факты берутся только из брифа.** Если данных мало, модель может обобщать. Такие места аудит помечает проверкой `facts_supported`, их можно исправить автоматически.
- **Растровые шаблоны восстанавливаются приближённо.** Переносятся фон, типографика, карточки и цвета. Логотипы и картинки исходника в новые колоды не попадают.
- **Иллюстрации генерируются по одной.** Личный тариф GigaChat не принимает параллельные запросы, поэтому на картинку уходит ~20–50 с.
- **Иллюстрации идут через хостинговый сервис.** Kandinsky вызывается через GigaChat API со своими условиями использования. Веса Kandinsky 3.x открыты (Apache 2.0), и генератор изолирован в `backend/app/layout/images.py`, но в этом прототипе они не развёрнуты локально. Подробности — в docs/MODELS.md.
- **Соединение с GigaChat без проверки TLS-сертификата.** Проверка отключена только для этого хоста, потому что сертификаты Минцифры могут отсутствовать в системе. Если это неприемлемо, установите корневой сертификат и уберите `verify=False` в `backend/app/layout/images.py` либо отключите иллюстрации.
- **Нет аутентификации и разграничения доступа** между пользователями: проекты и шаблоны общие для всех, кто дошёл до сервиса.
- **Только десктоп.** Мобильные устройства и планшеты не целевые платформы.

Карта репозитория

```
backend/app/
  parsing/     разбор шаблона: OOXML, шрифты, рендер, анализ слайдов, токены, холсты, паттерны,
растровые шаблоны
  planning/    бриф → контент-модель (LLM), сверка фактов, три варианта вёрстки
  layout/      композер рецептов, заполнение паттернов шаблона, подбор кегля, иконки, иллюстрации,
сборка PPTX
  audit/       детерминированные и контекстные проверки, каталог, исправления
  export/      HTML-просмотрщик
  llm/         OpenAI-совместимый клиент, реестр скиллов
  core/        общие утилиты работы с PPTX, замок PyMuPDF
  pipeline.py  оркестратор прогона и исправлений
  api/main.py  HTTP API и раздача фронтенда
  cli.py       командная строка
backend/skills/<имя>/<версия>.yaml   промпты скиллов и агентов (версионируются)
backend/assets/icons/lucide          иконки (лицензия ISC)
backend/tests/                       модульные и интеграционные тесты
backend/tools/                       отладочные скрипты разработчика
frontend/                            React 19 + Vite + Tailwind 4, токены VKUI, vk-bridge
examples/                            примеры брифов
docs/                                ARCHITECTURE, MODELS, AUDIT
config.yaml                          главный конфиг сервиса
Dockerfile, docker-compose.yml       контейнерный запуск
```

Архитектура

Документ описывает устройство системы: из каких слоёв она состоит, какие данные передаются между ними и почему решения приняты именно так. Как развернуть и пользоваться — в README. Модели — в MODELS.md, проверки аудита — в AUDIT.md.

Содержание

1. Идея и принципы
2. Общая схема
3. Данные между слоями
4. Слой 1. Разбор шаблона, стили и брендбук
5. Слой 2. Сценарий
6. Интерпретация данных
7. Слой 3. Три варианта и вёрстка
8. Слой 4. Аудит
9. Слой 5. Экспорт
10. Оркестрация: прогон, режимы, сроки, отказы
11. Модели и скиллы
12. Хранилище
13. HTTP API
14. Фронтенд
15. Как расширять систему
16. Принятые решения и компромиссы

---

1. Идея и принципы

Современные модели неплохо пишут содержание, но плохо воспроизводят дизайн-систему: ломают сетку, выдумывают шрифты и цвета, теряют логотипы. Поэтому «дизайнера» в сервисе делает не модель, а код, который читает чужой шаблон как набор правил. Модели пишут текст и оценивают смысл. Там, где модели доверена композиция слайда (дизайнерский вариант B), она работает в рамках той же дизайн-системы, а геометрию проверяет и чинит код.

Из этого следуют пять принципов.

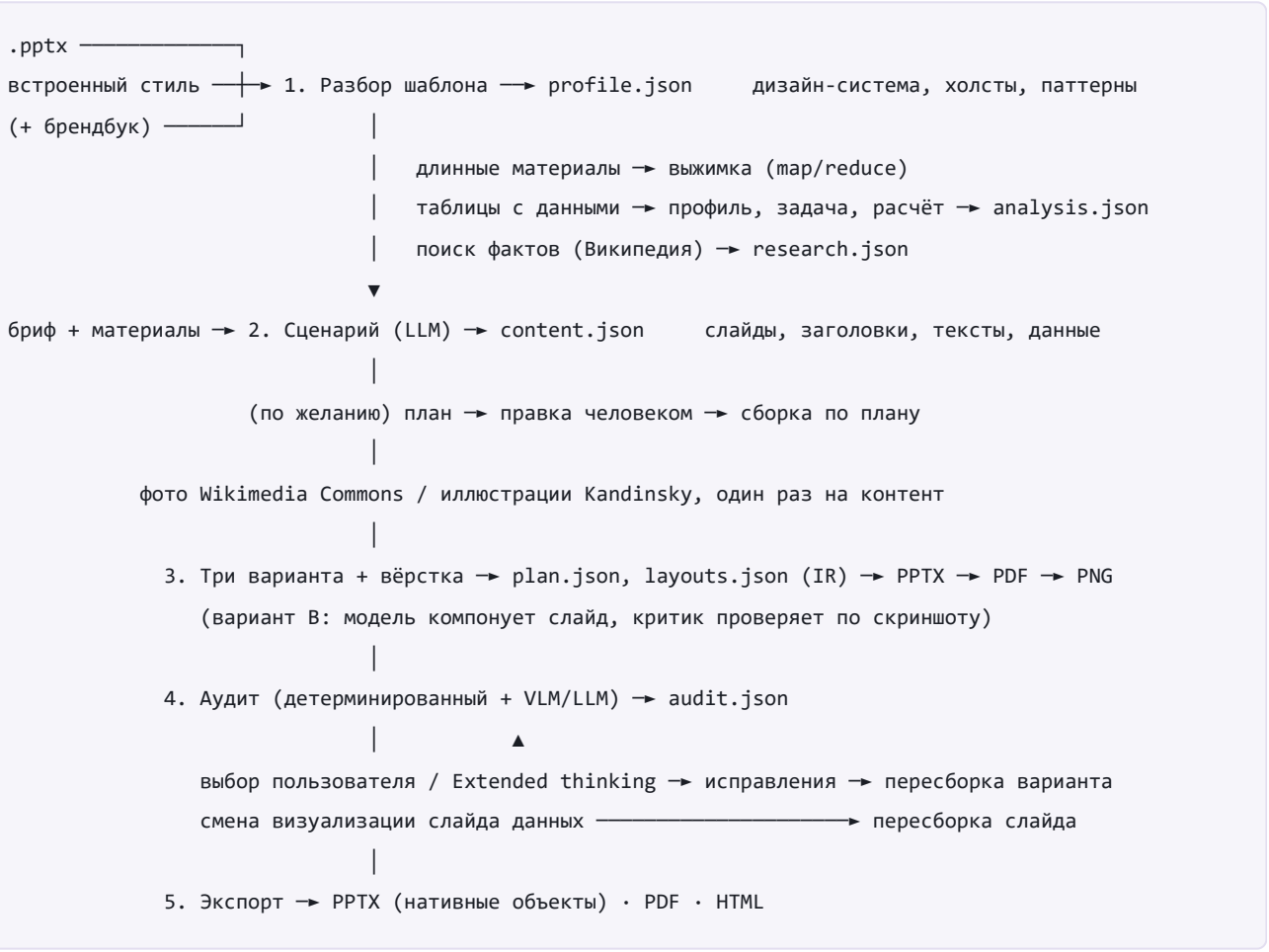
1. **Детерминизм прежде модели.** Размеры, цвета, шрифты, геометрия, подбор кегля и большинство проверок вычисляются без моделей. Числа из таблиц с данными тоже считает код, модель только формулирует вывод. Сбой модели не ломает вёрстку: разбор шаблона без VLM остаётся полным, а колода без контекстного аудита всё равно собирается и проверяется детерминированно.
2. **Гибрид редактирования шаблона и композиции.** Структурные слайды (титул, раздел, финал) и собственные композиции шаблона клонируются из оригинала: текст пишется прямо в его фигуры, форматирование абзацев сохраняется. Остальные слайды раскладывает композер на «холстах» — слайдах шаблона без контента. Результат собран из материала самого шаблона и не выдаёт генерацию.
3. **Явные артефакты между слоями.** Каждый слой получает и выдаёт типизированную модель (Pydantic) и пишет её JSON на диск. Любой шаг можно воспроизвести и проверить отдельно.

4. **Нативный результат.** PPTX собирается настоящими объектами (тексты, таблицы, диаграммы), а не картинками, и остаётся редактируемым.
5. **Аудит — часть пайплайна.** Проверки работают внутри сервиса, показывают зону замечания на слайде и умеют исправлять то, что пользователь отметил. В режиме Extended thinking исправимые замечания оформления правятся автоматически.

Оформление берётся из одного из двух источников: корпоративного шаблона `.pptx` или встроенного стиля Лекала. Встроенный стиль сам превращается в шаблон и дальше обрабатывается тем же путём. Брендбук (Figma, токены, PDF, логотипы) накладывается поверх любого из них.

Промпты не зашиты в код: это версионируемые файлы скиллов (см. раздел 11). Модель подключается по конфигурации.

2. Общая схема



Системные части:

Часть	Где	Роль
Слои обработки	`backend/app/parsing`, `planning`, `layout`, `audit`, `export`	предметная логика
Встроенные стили	`backend/app/styles`	каталог стилей, генератор `.pptx`-пака, подбор стиля по теме

Часть	Где	Роль
Брендбук	`backend/app/brand`	разбор `.fig`, токенов, PDF, картинок; применение к стилю и шаблону
Аналитика данных	`backend/app/analytics`	чтение таблиц, профиль датасета, задача, расчёт, рекомендации визуализации, сюжет колоды по данным
Поиск фактов и фото	`backend/app/research`	API Википедии и Wikimedia Commons, Яндекс Поиск API
Оркестратор	`backend/app/pipeline.py`	порядок слоёв, параллелизм, режимы и сроки, события прогресса, пересборка варианта
Клиент моделей и скиллы	`backend/app/llm/`, `backend/skills/`	вызовы моделей, версионируемые промпты, трасса
HTTP API	`backend/app/api/main.py`	шаблоны, стили, брендбук, материалы, проекты, SSE-прогресс, план, превью, исправления, скачивание, раздача фронтенда
Командная строка	`backend/app/cli.py`	тот же пайплайн без интерфейса
Веб-интерфейс	`frontend/`	мастер, прогресс, план, проект, аудит, визуализация, библиотека шаблонов
Конфигурация	`config.yaml`, `.env`	модели, пороги, режимы и сроки, варианты, версии скиллов, поиск, стили; секреты

Внешние программы: **LibreOffice** (рендер PPTX → PDF) и **PyMuPDF** (PDF → PNG и SVG; работает под общим замком, потому что не потокобезопасен). Для данных и брендбуков добавлены **pandas** и **openpyxl** (таблицы CSV/XLSX) и **zstandard** (блок данных `.fig`).

3. Данные между слоями

Слои обмениваются только этими моделями:

Модель	Файл	Что содержит	Кто пишет → кто читает
`TemplateProfile` (`parsing/model.py`)	`profile.json`	палитра, типографика, шкала кеглей, поля и сетка, стили карточек и таблиц, холсты, паттерны, слайды-образцы, нарратив, стиль иллюстраций; источник (`source`: загруженный шаблон или стиль), `style_id`, моноширинный шрифт	разбор → сценарий, вёрстка, аудит

Модель	Файл	Что содержит	Кто пишет → кто читает
`Brief` (`planning/models.py`)	в `job.json`	тема, назначение, аудитория, данные, число слайдов, длительность, язык, режим иллюстраций; встроенный стиль и свои цвета, приложенные материалы целиком, поиск фактов, датасеты и цели анализа, брендбук и его приоритет, источник картинок, режим генерации	пользователь → сценарий
`BrandKit` (`brand/model.py`)	`brands/<id>/brand.json`	цвета с ролями, акценты, шрифты по ролям, логотипы, примеры слайдов, правила, отступы, минимальный размер и охранное поле логотипа, предупреждения	брендбук → стиль или производный шаблон
`Insight` (`analytics/analyze.py`)	`analysis.json`	по датасету: природа, профиль `DataProfile`, задачи с баллами, агрегат, факты и ключевые цифры, вывод по умолчанию, 2–3 варианта визуализации, выбранная	аналитика → сценарий, смена визуализации
`Research` (`research/web.py`)	`research.json`	запросы, прочитанные статьи, факты с номерами источников, источники	поиск → сценарий, подбор фото
`DeckContent` (`planning/models.py`)	`content.json`	слайды с намерением (`intent`), заголовком, пунктами, диаграммой, таблицей, колонками, цитатой, кодом, промптом и запросом картинки, подписью фото и источника, блоком «и что?», данными слайда-аналитики, заметками; список фактов и источников; целевой объём	сценарий → варианты, аудит
`Outline` (`planning/outline.py`)	— (через API)	план колоды: тип, заголовок, тезисы и пожелание к каждому слайду	сценарий → человек → сценарий
`VariantPlan` (`planning/models.py`)	`plan.json`	для каждого слайда: рецепт, холст, тон, запасные холсты	варианты → композер
`SlideLayout` (`layout/ir.py`)	`layouts.json`	промежуточное представление вёрстки слайда: заполнение слотов, элементы `El` в координатах слайда (EMU), заметки, предупреждения	композер / дизайнер → сборщик, аудит, исправления
`AuditReport` / `Issue` (`audit/model.py`)	`audit.json`	замечания: код проверки, важность, слайд, зона `bbox`, признак исправимости	аудит → интерфейс, исправления
`JobState` (`pipeline.py`)	`job.json`	статус (`queued`, `running`, `outline`, `done`, `error`), режим (`direct` или `outline`), выбранные варианты, этап, события, варианты, версии скиллов, статистика вызовов	оркестратор → API

Намерения слайда (`intent`): `title` , `agenda` , `section` , `bullets` , `cards` , `process` , `timeline` , `stats` , `chart` , `table` , `comparison` , `quote` , `image` , `code` , `closing` .

Новые поля слайда (`ContentSlide`): `code` — листинг (`lang` , `text` , `caption`); `image_query` и `image_credit` — запрос фото и подпись «автор, лицензия»; `source_note` — источник цифр для подвала; `data` — датасет, задача, выбранная визуализация и варианты для слайда данных; `why` / `then` с заголовками `why_head` / `then_head` — блок «почему это важно / что из этого следует»; `instruction` — пожелание человека, которое сценарист учитывает при дописывании. У диаграммы (`ChartSpec`) появились типы `scatter` , `stacked_column` , `stacked_bar` , подсветка категорий, подписи осей, формат чисел, границы оси, точки и линия тренда для рассеяния.

Промежуточное представление (`SlideLayout`) отделяет решения вёрстки от OOXML: композер, дизайнер варианта B и исправления работают с ним, и только сборщик знает про формат PPTX. У элемента `El` есть отражение линии (`flip_h` , `flip_v`) и пунктир (`dash`): дизайнер рисует ими стрелки «назад» и схемы.

4. Слой 1. Разбор шаблона, стили и брендбук

`backend/app/parsing/` . Цель: превратить чужой `.pptx` в `TemplateProfile` — набор правил, по которому можно строить новые слайды.

Стадии разбора (`profile.py`)

Прогресс	Стадия	Что происходит
5%	Шрифты шаблона	извлекаются встроенные шрифты
12%	Структура слайдов	`TemplateAnalyzer` разбирает каждый слайд; недостающие шрифты подкачиваются из Google Fonts и регистрируются для рендера
20%	Рендер шаблона	LibreOffice → PDF → PNG превью
30%	Слайды-картинки	только для растровых шаблонов: дизайн-система восстанавливается по пикселям
50%	Дизайн-токены	палитра, типографика, сетка, стили карточек и таблиц
60%	Холсты	слайд минус контент, с рендером каждого холста
70%	Паттерны шаблона	слайды-образцы, пригодные для клонирования
80%	Разметка моделью	VLM описывает стиль, тон и стиль иллюстраций (необязательно)
100%	Готово	профиль сохранён

Результат кэшируется по sha256 файла: повторная загрузка того же шаблона мгновенна. Ключ `--force` (CLI) или поле `force` (API) разбирает заново. Разбор принимает и свой идентификатор с готовыми полями профиля (`tid` , `extra`): так разбираются паки встроенных стилей и производные шаблоны брендбука, у которых хэш файла нестабилен или палитра известна заранее.

Модули

Модуль	Что делает
`ooxml.py`, `extract.py`	Фигуры с абсолютными координатами (с учётом групп), наследование стилей run → lstStyle → макет → мастер, цвета темы и `clrMap`, заливки
`fonts.py`	Встроенные шрифты (EOT/обфусцированные) → TTF, подкачка из Google Fonts, инстансы вариативных шрифтов, метрики через FreeType. Шрифт без кириллицы меряется запасным шрифтом рендерера, отсутствующий жирный — с поправкой на синтетическое утолщение
`render.py`	LibreOffice headless с отдельным профилем на вызов → PDF → PNG. Шрифты шаблона регистрируются в сессии (Windows GDI / fontconfig). PyMuPDF — под общим замком
`analyze.py`	Для каждого слайда: заголовок, кикер, плашка заголовка; «хром» (фон, логотипы, колонтитулы, линейки, повторяющиеся элементы) против контента; служебные слайды (инструкции, палитры, библиотеки иконок); вид слайда (титул / раздел / план / контент / финал); **повторяющиеся группы** — карточки в подложках или, без подложек, наборы текстов одного стиля, выстроенные в ряд, колонку или сетку, вместе с иконками. Колонтитул, который шаблон явно пометил именем фигуры (`footer…`), становится слотом `footer`: туда пишется название колоды или подпись источника
`tokens.py`	Палитра (цвета, взвешенные по площади и объёму текста; роли: фон, текст, основной, акценты, цвета диаграмм), типографика по ролям, шкала кеглей, сетка и шаг, стили карточек и таблиц
`canvases.py`	Холсты: слайд минус контент. По пикселям рендера уточняются свободная область, фон под слотами и ширина заголовка — логотипы бывают «вшиты» в картинку фона
`patterns.py`	**Паттерны шаблона.** Слайды-образцы с карточками, строками, крупными цифрами, таблицами, диаграммами или картинкой, пригодные для клонирования. Образец отбрасывается, если после заполнения на нём останется чужой контент: несколько скриншотов, декоративные рамки устройств, большие удаляемые тексты
`raster.py`	Шаблоны из слайдов-картинок (экспорт Figma/Canva). Гладкий фон восстанавливается аппроксимацией от рамки слайда. Находятся строки заголовков, их цвет и кегль, белые карточки и акценты. Из этого собирается редактируемый шаблон-заменитель и разбирается обычным путём
`vlm_labels.py`	VLM (скилл `template_analyst`) по контактному листу описывает стиль, тон и стиль иллюстраций. Детерминированный профиль остаётся первичным, модель только дополняет
`profile.py`	Оркестратор разбора и кэш профилей

Что попадает в профиль

- **Палитра:** цвета с весами и ролями, светлый и тёмный фон, цвета текста, основной цвет, акценты, цвета диаграмм, подложка карточек. `Palette.allowed()` — список разрешённых цветов для аудита.
- **Типографика:** гарнитуры заголовка и текста, допустимые кегли, стили ролей (`title`, `subtitle`, `kicker`, `heading`, `body`, `caption`, `number`).
- **Сетка:** типовая контентная область (поля) и шаг.
- **Холсты** (`Canvas`): вид (титул, раздел, финал, план, контент), слоты для текста с координатами, стилем и ёмкостью в символах, контентная область, фон, тёмный или светлый, «чистый» ли холст, оценка пригодности, превью.
- **Паттерны** (`PatternInfo`): вид (карточки, строки, цифры, таблица, диаграмма, картинка), раскладка, число пунктов, роли текста в пункте, что удалить при заполнении, куда можно вставить иллюстрацию.

- **Нарратив шаблона:** есть ли план и разделители, заглавные ли заголовки, есть ли кикеры, нумерация, средняя длина заголовка, тип финала. Планировщик ему следует, аудит его учитывает: если шаблон пишет короткие заголовки-темы, «нет вывода в заголовке» — только совет.

Уточнения разбора на трудных шаблонах

- **Подписи под фото не становятся заголовком.** На слайдах вида «команда» самый крупный текст — имя под фотографией. Контентный холст отбрасывается, если его слот заголовка уже 15% ширины слайда и вмещает меньше 12 символов: заголовку-выводу там не уместиться (`canvases.py: draft_canvases`).
- **Неоновая палитра тёмных шаблонов.** Если среди цветов нет насыщенного средней светлоты, основным цветом может стать яркий насыщенный (насыщенность > 0,35, светлота ≥ 0,75): лайм или бирюза на тёмном фоне — тоже фирменный цвет (`tokens.py: build_palette`).
- **Градиентные заливки таблиц.** Заливка ячейки вида «градиент» приводится к сплошному среднему цвету, картинка — к отсутствию заливки (`tokens.py: solid_fill`). Тем же правилом пользуется рецепт таблицы для профилей, разобранных раньше.
- **Контраст серий диаграмм.** Цвет серии, почти сливающийся с фоном холста (контраст < 1,6: тёмная серия на тёмном слайде), заменяется акцентом шаблона с контрастом ≥ 2, а не пропадает с диаграммы; серий в палитре всегда не меньше трёх (`layout/style.py: Look._chart_colors`).

Растровые шаблоны

Если слайды шаблона — картинки без фигур (экспорт из Figma или Canva), `is_raster` это распознаёт. Тогда из пикселей восстанавливается редактируемый шаблон-заменитель: фон, строки заголовков с цветом и кеглем, карточки и акценты. Он разбирается тем же путём, что и обычный шаблон. Логотипы и картинки исходника в новые колоды не переносятся.

Встроенные стили Лекала

`backend/app/styles/`. Если корпоративного шаблона нет, колода строится во встроенном стиле.

- **Каталог** — 9 стилей в `styles/catalog/*.yaml` (`StyleSpec`): категория (работа, финансы, промо, учёба, консалтинг), настроение, палитра по ролям (фон, подложка, основной, вторичный, акцент, текст, приглушённый, линия, тёмный фон и текст на нём), шрифты заголовка, текста и кода, хром (мотив, вид обложки) и вариант генератора (`editorial`, `bold`, `minimal`, `academic`).
- **Пак** (`generator.py`). Из стиля генерируется обычный `.pptx` из семи слайдов: обложка, план, раздел, три контентных образца, финал. Сетка 960×540 «пикселей» (1 px = 12 700 EMU), поля 64 px, кикер, заголовок, тонкая линия, подвал с номером. Пак проходит через тот же парсер, поэтому рецепты, аудит, исправления и экспорт работают со стилями без изменений. Палитра в профиль записывается из стиля, а не угадывается по площадям (`spec_palette`); в профиле `source = "style"` и моноширинный шрифт для блоков кода.
- **Кэш.** Идентификатор пака — `style-<id>-<хэш описания и версии генератора>`; пак собирается один раз (под замком на идентификатор), дальше берётся из кэша. Первая сборка стиля занимает порядка 10–30 секунд.
- **Свои цвета.** Пользователь задаёт фон, основной, вторичный и вспомогательный цвет. Производные (текст, приглушённый, линии, подложка, тёмный фон) пересчитываются от них (`StyleSpec.with_overrides`).
- **Авто-подбор.** Скилл `style_picker` называет три подходящих стиля по теме, назначению и аудитории; берётся первый, которого не было в трёх последних колодах в стилях Лекала, — иначе все учебные

презентации выходили бы в одном стиле. Без ответа модели стиль выбирается по назначению брифа.

В библиотеке шаблонов компании паки стилей не показываются.

Брендбук

`backend/app/brand/`. Брендбук приходит в любом сочетании файлов (`POST /api/brand`, до 10 файлов по 200 МБ) и сводится в один `BrandKit`.

Источник	Модуль	Что извлекается
`.fig` (Figma, «Save local copy»)	`fig.py`, `kiwi.py`	Собственный разбор формата без Figma и без сети: ZIP → `canvas.fig` (заголовок `fig-kiwi`, схема kiwi и блок данных deflate/zstd) → плоский список узлов. Цвета — из стилей заливки с именами, переменных-цветов и реальных заливок (вес — площадь); шрифты — из текстовых стилей и текстов макетов (вес — число символов); логотипы — узлы с «logo/лого» в имени: вектор собирается в SVG из геометрии и переводится в PNG, растр берётся из `images/`; примеры слайдов — фреймы 16:9 верхнего уровня
JSON дизайн-токенов	`tokens.py`	W3C Design Tokens (`$value` / `$type`), Tokens Studio (`value` / `type`), плоский словарь `{"colors":…, "fonts":…}`; ссылки вида `{colors.primary}` разрешаются; отступы и правила логотипа
PDF-гайдлайн	`pdf.py`	Цвета из текста (HEX, RGB, CMYK) с подписями, шрифты документа и упомянутые в тексте, палитра по рендеру страниц (вес — площадь), логотип — крупная картинка с прозрачностью, правила — строки с запретами и требованиями («нельзя», «охранное поле», «минимальный размер»)
PNG/SVG/JPG, ZIP	`images.py`	Логотипы (по имени файла или прозрачности) и примеры слайдов; палитра по площади цветов

Сведение (`store.py`). Несколько файлов дополняют друг друга: цвета и шрифты берутся из самого точного источника (`.fig` и токены точнее PDF, PDF точнее картинок), логотипы — откуда нашлись (до четырёх; приложенные отдельными файлами идут первыми), правила — из PDF. Роли цветов (основной, вторичный, акценты, фон, текст, приглушённый) назначает `assign_roles`: по именам («primary», «brand», «фон»…), по весу использования, по насыщенности и светлоте. Брендбук кэшируется по хэшу содержимого файлов и версии разборщика и хранится в `data/brands/<id>/`.

Применение (`apply.py`).

- **Встроенный стиль:** цвета ролей, шрифты, логотипы для светлого и тёмного фона и сетка (поля, скругление, минимальная высота логотипа, охранное поле) передаются в `StyleSpec.with_overrides`; собирается свой пак. Свои цвета пользователя применяются поверх брендбука.
- **Шаблон компании:** собирается производный шаблон — копия `template.pptx`, в которой тема (`a:clrScheme` accent1–6 и `a:fontScheme`) получает цвета и шрифты бренда; явные цвета фирменных оттенков на слайдах, макетах и мастере заменяются ролями бренда (нейтральные — фон, текст — не трогаются); шрифты заменяются, если шрифт бренда есть в системе (иначе — предупреждение); картинки на месте логотипа шаблона заменяются логотипом бренда с сохранением пропорций. Затем производный шаблон проходит обычный разбор, и дальше пайплайн не отличает его от загруженного. Идентификатор — `<шаблон>-brand-<брендбук>-b|t`; в библиотеке шаблонов такие профили не показываются.

Приоритет (`Brief.brand_priority`). По умолчанию (`brand`) брендбук важнее шаблона: меняются цвета, шрифты и логотип. При `template` («важнее шаблон») от брендбука остаются только логотипы.

Предупреждения (логотип шаблона не найден, шрифта нет в системе) уходят в событие этапа `brand` и показываются в ходе генерации.

5. Слой 2. Сценарий

`backend/app/planning/`, `backend/skills/`. Цель: по брифу получить `DeckContent` — слайды с заголовками, текстом, данными и заметками спикера. Слой ничего не знает о координатах и OOXML.

Ход планирования (`planner.py: plan_deck`)

1. **Объём.** Пользовательское число слайдов, иначе при заданной длительности выступления ≈1,5 слайда на минуту (в пределах 10–15), иначе при длинном материале (больше 15 тыс. знаков) — 15–20 слайдов по его размеру, иначе 11–14 по богатству брифа.
2. **Скилл `deck_planner`** (v8). Получает бриф, сюжетную арку под назначение (фича, продукт, проект, инициатива, обучение, другое), формат повествования шаблона, ограничения шаблона (длина заголовка, кикеры, число разделов), каталог иконок, правила иллюстраций и заметок. Ответ — JSON с починкой при ошибке разбора.
3. **Нормализация** (`normalize`, `norm_slide`). Ответ модели приводится к моделям слайдов. Числа не из брифа и приложенных материалов отбрасываются; диаграмма строится только если значения найдены в источнике; таблица — только со строками, чьи числа есть в источнике; слайды с недостающими данными переводятся в более простой вид (`chart` → `stats` → `cards`, `table` → `cards`). Пункты колонок, пришедшие объектами, превращаются в текст; листинг кода сохраняется построчно с отступами (до 18 строк).
4. **Титул и финал.** Гарантируются всегда: недостающие добавляются.
5. **Разделители.** Ограничиваются нарративом шаблона: не больше 3 при 15+ слайдах, 2 при 9–14, иначе нет.
6. **Добор.** Если слайдов меньше нужного, скилл `deck_extender` дописывает недостающие отдельным коротким вызовом: без повторов тем и пустых слайдов, без титулов, разделителей и цитат.
7. **Обрезка.** Лишние слайды сверх лимита убираются в порядке: цитаты, разделители, содержательные слайды с конца; титул и финал остаются.
8. **Разнообразие** (`diversify`). Колода из одних карточек читается как шаблон: карточек, как и списков, — не больше трети содержательных слайдов, «плоский» тип (карточки, список, цитата, картинка, цифры) не повторяется два раза подряд. Лишний слайд получает другой тип по смыслу (три и больше годов → таймлайн, противопоставление → сравнение, иначе — реже всего встречавшийся) и указание сценаристу переписать его с сохранением фактов. Слайды с диаграммой, таблицей и кодом не трогаются.
9. **Слайды с кодом** (`ensure_code`, `planning/code.py`). Если в материалах есть код, а в колоде нет ни одного листинга, добавляются 1–2 слайда `code`: фрагменты находятся по виду строк, собираются с подписью («Листинг 1. …»), для Python отступы восстанавливаются по двоеточиям. Слайд встаёт рядом с самым близким по словам, код переносится дословно, сценарист пишет только заголовок-вывод и пояснения.
10. **Тонкие слайды** (`enrich_thin`). Содержательный слайд с одним-двумя тезисами или почти без текста дописывается по брифу и материалам — одним вызовом `slide_writer` вместе со слайдами, сменившими тип на шаге 8.
11. **Иллюстрации.** Скилл `illustrator` (v2) пишет буквальные сцены по теме колоды и тезисам слайда и короткий английский запрос для поиска фото. Режим «обязательно» гарантирует 2–3 слайда с картинкой, «где уместно» — хотя бы одну при 9+ слайдах.

12. **Заметки спикера.** Объём считается от длительности (130 слов в минуту). Пустые и слишком короткие заметки дописывает скилл `speaker_notes`.

Материалы

К брифу можно приложить файлы — txt, md, pdf, docx, pptx и другие текстовые (до 12 файлов за раз). `planning/materials.py` извлекает текст: до 150 тыс. знаков на файл и до 200 тыс. в сумме. Таблицы (CSV/TSV, XLSX/XLS, табличный JSON и таблицы внутри txt/md) становятся датасетами — см. раздел 6. Текст материалов целиком хранится в `Brief.source_text`: по нему сверяются числа.

Длинные материалы (`planning/digest.py`). Материал до 25 тыс. знаков (`materials.digest_over_chars`) идёт в бриф целиком. Длиннее — читается целиком по кускам: текст режется по строкам на фрагменты ≈8 тыс. знаков (кусок закрывается на заголовке), каждый конспектирует скилл `material_digest` параллельно (разделы, тезисы, термины, этапы, примеры, факты), конспекты сводит `material_merge` в выжимку до 12 тыс. знаков. Строки выжимки с числами, которых нет в исходном тексте, выбрасываются. Сбой одного куска не отменяет остальные.

Факты из Википедии (`research/`)

Галочка «Факты из интернета» (`Brief.web_search`) запускает исследование темы до планирования. Источник задаётся в `config.yaml → search.provider`: по умолчанию бесплатный API Википедии, по настройке — Яндекс Поиск API (генеративный ответ с источниками, платно), `none` — выключено.

1. Скилл `web_researcher` формулирует запросы (4 на колоду).
2. Полнотекстовый поиск и поиск по названиям статей; нужные статьи из выдачи выбирает скилл `wiki_picker` (без ответа модели — по совпадению слов).
3. До 4 статей читаются целиком (2, если пользователь сам приложил подробный материал больше 10 тыс. знаков: тогда из сети берётся только дополнение — история, применение, примеры).
4. Скилл `wiki_reader` выписывает факты (до 8 со статьи); факт с числом, которого нет в статье, отбрасывается.
5. Факты с номерами источников добавляются в бриф блоком «ДАННЫЕ ИЗ ИНТЕРНЕТА» — их числа становятся «числами брифа» для сверки.
6. После сценария `attach_sources` находит слайды, где стоит характерное число из найденного факта: источник пишется в подвал слайда, ссылки — в заметки спикера; использованные источники сохраняются в `DeckContent.sources`.

Сбой поиска не роняет прогон: колода строится по брифу. Результат лежит в `research.json`, главные картинки прочитанных статей становятся кандидатами в фото.

Уточняющие вопросы и план перед сборкой

- **Уточнения** (`POST /api/clarify`). Если материал большой (больше 15 тыс. знаков) или бриф совсем короткий, перед запуском показываются до двух вопросов с вариантами ответа. Вопрос об объёме задаётся по правилу (≈12, ≈16 или ≈20 слайдов); остальные пишет скилл `clarifier`, ответ может менять только аудиторию, глубину подачи и акцент. Без модели остаются только вопросы по правилам.
- **План** (`planning/outline.py`). Если выбрано «Сначала показать план» (по умолчанию — для материала больше 15 тыс. знаков), прогон останавливается после сценария в статусе `outline`. Человек правит заголовки, порядок, типы и тезисы слайдов, добавляет и удаляет слайды, пишет пожелание к слайду («сделай таблицей»). По `POST /api/jobs/{id}/build` изменённые и новые слайды дописывает скилл

`slide_writer` одним вызовом (листинги кода не переписываются, числа сверяются с источником), неизменённые остаются как есть, и прогон продолжается с готовым контентом.

Сверка с брифом (`planning/facts.py`)

Каждое число на слайдах — проценты, суммы, количества, сроки, годы — ищется в брифе и приложенных материалах. Исключения: нумерация («Шаг 2», «01») и очевидные производные в явном обороте («в 3 раза» при «с 4,5 до 1,5 ч»). Список `facts`, который возвращает сам планировщик, источником не считается. Слайды с неподтверждёнными числами до сборки отправляются корректору (`fixer`): он убирает число или заменяет его качественной формулировкой. Так «первые 4 склада» при плане в 12 или «Q3 2026» без дат в брифе не доходят до колоды.

Сокращение текста

Если текст не влез в макет, `text_condenser` сокращает его под лимит символов конкретного слота (шаг `condense` в оркестраторе).

6. Интерпретация данных

`backend/app/analytics/`. Цель: по приложенным таблицам получить слайды, которые отвечают на вопрос «что показывают данные и что из этого следует». Код читает таблицы, определяет природу датасета, выбирает задачу и визуализацию и считает все числа; модель только формулирует вывод по уже посчитанным фактам, и её числа сверяются с ними.

Путь датасета

Шаг	Модуль	Что происходит
Приём	`ingest.py`	CSV/TSV (разделитель — тот, что даёт одинаковое число колонок в первых строках; кодировка UTF-8 или cp1251), XLSX/XLS/XLSM (каждый лист), табличный JSON, таблицы внутри txt/md. Из файла получается один или несколько датасетов (`Table`) и текст вокруг таблиц — он идёт сценаристу как контекст. Исходник сохраняется в `data/uploads/`
Профиль	`analyze.py: profile, describe`	`DataProfile`: тип колонки (число, дата, категория, текст, идентификатор) и роль (измерение, метрика, время, идентификатор, текст); годы — измерение, а не метрика; пропуски по колонкам, доля пустых ячеек и полных строк; выбросы по IQR (за пределами 1,5 межквартильного размаха, при 8+ значениях); диапазоны и медианы; ось времени и её шаг (внутри дня, день, неделя, месяц, квартал, год); иерархии измерений (каждому значению A соответствует одно значение B, у B 2–30 значений: город → регион)
Развёртка	`wide_to_long`	Широкая таблица с периодами в заголовках («Продукт · I кв. · II кв. …») разворачивается, и периоды становятся осью времени
Задача	`recommend.py: choose_tasks`	Все выполнимые задачи с баллами и объяснением: динамика, сравнение, структура (доли), распределение, взаимосвязь, справочные значения, качество данных. Природа данных даёт +3 (справочник +2), цель анализа +3,5 той задаче, о которой она говорит. Цель берётся из карточки датасета или из предложения брифа, которое лучше всего совпало с его колонками (`assign_goals`). Журнал событий (много строк в день, меньше полугода) — распределение, а не динамика; таблица с пропусками ≥10% в колонке — сначала качество данных

Шаг	Модуль	Что происходит
Расчёт	`analyze.py: _timeseries, _comparison, …`	Агрегат, факты, ключевые цифры и вывод по умолчанию. Все числа форматируются один раз в русской записи — сверка находит их на слайде дословно. Длинные ряды агрегируются (по месяцам), больше 12 категорий — первые 11 и «Прочие», мелкие доли объединяются
Визуализация	`recommend.py: recommend, build_chart`	2–3 варианта по убыванию балла, у каждого объяснение «почему», у первого — почему не подошли остальные типы
Перекрёстные факты	`cross.py`	Между датасетами — только там, где совпадают ключи: одно и то же измерение (город в выручке и в анкете) → доли одного ключа в обоих; общая ось времени → корреляция по общим периодам; до 4 фактов
Слова и сюжет	`story.py`	Скиллы `data_analyst` (слайд на датасет) и `data_story` (сюжет колоды); сверка каждого числа с фактами кода

Нераспознанный файл (меньше двух строк; ни чисел, ни повторяющихся категорий; нет таблиц с заголовками колонок) в колоду не попадает: в форме его карточка помечена «не распознано» с причиной, в событии этапа `data` он тоже виден с причиной.

Выбор визуализации

Типы: линейный график, столбцы, сгруппированные столбцы, горизонтальные полосы, гистограмма, кольцевая и круговая, рассеяние с линией тренда (МНК), накопительные столбцы и полосы, таблица, карточки с ключевыми цифрами. Правила по задаче:

- **динамика:** больше 8 точек — линия (сезонность при 24+), до 8 или кварталы и годы — столбцы; несколько рядов одной единицы, складывающихся в итог, — накопительные столбцы; круговая не подходит — периоды не части целого;
- **сравнение и справочник:** длинные подписи (больше 10 символов) или больше 7 категорий — полосы, иначе столбцы; итог из нескольких частей — накопительные полосы; кольцо — только если значения складываются в целое и категорий не больше 6;
- **доли:** кольцо или круговая (круговая — пока секторов не больше пяти), полосы — когда доли нужно точно сравнить;
- **распределение:** гистограмма, альтернатива — медиана, квартили и 90-й процентиль карточками;
- **взаимосвязь:** рассеяние с линией тренда, альтернатива — средняя по группам столбцами;
- **качество данных:** полосы полноты по колонкам, худшая — сверху.

Таблица предлагается, только если в ней не больше 7 строк; если цель просит «таблицу» или «точные значения», её балл растёт. Тип не зависит ни от формата файла, ни от шаблона: `build_chart(insight, type)` собирает любой вариант из одного и того же агрегата, поэтому переключение визуализации не требует пересчёта.

Слова модели и сверка чисел

Скилл `data_analyst` (v2) для каждого датасета пишет заголовок-вывод, вводную фразу, подписи ключевых цифр, блок «почему это важно / что из этого следует» с короткими заголовками-мыслями и текст спикера. Скилл `data_story` для колоды по данным пишет главную мысль, порядок датасетов и разделы сюжета, связки между слайдами, три ключевые цифры, выводы, рекомендации и финал.

Каждый текст модели проходит сверку (`story.py`):

- числа — только из фактов кода, ключевых цифр и вывода по умолчанию (годы 1990–2035 считаются контекстом периода); «ускорение» и «замедление» — только если они есть в фактах;
- причинные формулировки без доказательства («за счёт», «благодаря», «доказывает», «напрямую зависит») и пустые фразы («требует внимания», «важно учитывать») отбрасываются;
- подпись ключевой цифры берётся только у пункта с тем же значением и не повторяет саму цифру.

Не прошло — подставляется текст кода. **Заметки спикера** проверяются по предложениям: неподтверждённое предложение выпадает само, а не уносит весь текст; числа профиля датасета (строки, выбросы, точки, метод расчёта) тоже посчитаны кодом и считаются подтверждёнными. Если подтверждённых предложений меньше двух, в заметки идут факты кода. В конце заметок — профиль датасета, метод и почему выбрана диаграмма.

Колода по данным

Если датасетов два и больше или кроме данных нет других материалов, каркас колоды строит код, а модель пишет слова (`story.data_deck`):

титул («Анализ данных: N датасетов») → **«Главное»** (главная мысль и три цифры, каждая дословно — ключевая цифра одного из датасетов, плюс «почему важно / что следует») → **план** (разделы сюжета с главным выводом каждого и пункт «Выводы и решения») → вступление-вопросы (при 4 датасетах и меньше, если модель его дала) → **слайды данных** в порядке сюжета → **выводы** → **рекомендации** → **финал**.

Порядок датасетов предлагает `data_story` ; кого модель не назвала, идут в конце в исходном порядке (динамика → сравнение → доли → распределение → связи → справочник → качество данных). Методика — профиль каждого датасета и почему выбрана диаграмма — пишется в заметки последнего слайда данных; отдельного слайда-таблицы о файлах и слайда с ошибкой разбора нет. Без ответа модели выводы собираются из заголовков-выводов датасетов.

Если датасет один и есть другие материалы, колоду пишет обычный сценарист: ему передаются посчитанные факты и указание не делать слайдов с диаграммами по этим датасетам, а слайды данных вставляются после вступления.

Слайд данных

- Намерение — `chart` , `table` или `stats` в зависимости от выбранной визуализации; заголовок — вывод с числом.
- Подписи осей с единицами, формат чисел, границы оси (проценты — до 100); главное (лидер, пик, худшая колонка) — основным цветом, остальное — приглушённым; на длинных рядах подписи значений снимаются.
- Ключевые цифры — сбоку от диаграммы; источник («Источник: файл») — в подвал слайда или мелкой строкой под контентом.
- **Блок «и что?»** (`recipes.py: takeaway`) — плашка «почему это важно / что из этого следует» под контентом. Если не помещается, она сжимается до компактной, затем до одной строки «что следует», а если не помещается и она — остаётся в заметках. Подпись второй части выбирается по первому глаголу: «Что проверить», «Что сделать», «Чего избегать», «Решение».
- Много категорий с тяжёлым хвостом (больше 12 категорий, хвост за первыми 10 даёт больше 25% итога): вторая часть списка (категории 11–30) — отдельным слайдом-продолжением, а не строкой «Прочие».

Слайды с блоком «и что?» верстает рецепт: паттерны шаблона и дизайнер варианта B их не берут — у образца нет места под вывод.

Переключение визуализации

На странице проекта у слайда данных есть панель «Визуализация» (`frontend/src/components/VizPanel.tsx`): задача, файл, 2–3 варианта с объяснением, отметка «рекомендовано» и почему не подошли остальные типы. Выбор варианта вызывает `POST /api/jobs/{id}/variants/{vid}/slides/{n}/chart` с `{type}`, и `pipeline.switch_chart`:

1. проверяет, что тип был среди предложенных для этого слайда;
2. берёт расчёт датасета из `analysis.json` и применяет новый тип к слайду (`story.apply_viz`): диаграмма, таблица или карточки;
3. заново подбирает рецепт и холст под новое намерение и компонует один слайд;
4. сохраняет `content.json` и вызывает общую пересборку варианта `rebuild_variant` с изменённым слайдом.

`rebuild_variant` собирает PPTX, PDF и PNG, повторяет детерминированный аудит целиком, а контекстный — только по изменённым слайдам, обновляет HTML и увеличивает номер ревизии. Той же функцией пользуются исправления по выбору пользователя и автоисправление в режиме Extended thinking. Продолжение длинного списка отдельно не переключается.

7. Слой 3. Три варианта и вёрстка

`backend/app/planning/variants.py`, `backend/app/layout/`.

Три варианта

Одна контент-модель раскладывается тремя стратегиями. Ось различий — **плотность × способ визуализации**. На этих двух параметрах пользователи чаще всего расходятся с автором шаблона, и они не требуют повторной генерации текста: варианты различимы визуально, но одинаково соответствуют правилам шаблона.

	Стратегия	Что меняется
A · Классический	`faithful`	Максимум паттернов шаблона, баланс текста и визуала, основной холст шаблона (изредка второй по популярности, как в самом шаблоне)
B · Дизайнерский	`visual`	Каждый содержательный слайд компонует модель по дизайн-системе — свои схемы, шевроны, сетки и акценты; затем проверяет себя по скриншоту. План варианта строится по визуальной стратегии (крупные цифры, диаграммы, иконки, чередование тёмных и светлых холстов) без паттернов шаблона
C · Компактный	`dense`	Слияние коротких соседних слайдов, таблицы и сравнения вместо карточек, строки вместо карточек, без разделителей; объём при этом не ниже минимума

Соответствие «намерение → рецепт» для трёх стратегий задано таблицей `RECIPES` в `variants.py`. Например, `bullets` даёт в A маркированный список, в B карточки, в C строки; `comparison` в C превращается в таблицу; `code` во всех вариантах — листинг.

Описание вариантов — в `config.yaml → variants`, поэтому число и состав вариантов настраиваются. Можно собрать только один вариант (`variants` в `POST /api/jobs`; в форме — нажатие на карточку A, B

или C): так быстрее.

Выбор основы слайда

1. `PatternPicker` оценивает пригодность каждого паттерна шаблона для слайда: вид, число пунктов, ёмкость слотов, наличие картинки. Он штрафует повторы и учитывает стратегию варианта (B любит цифры и картинки, C — таблицы и строки). Если паттерн набрал меньше порога, он не используется. Образец не берётся, если у слайда есть блок «и что?», если в образце больше 50 фигур (инфографика своей темы) или если заголовки пунктов вместе с текстом не влезут в карточку.
2. `CanvasPicker` выбирает холст, если паттерн не подошёл: по виду слайда, тону и стратегии; для запасных вариантов держит список альтернатив (если на основном не влезает заголовок).
3. Если и холст не подошёл, слайд собирает композер по рецепту.

Вёрстка

- **Композер** (`recipes.py`) раскладывает рецепт в контентной области холста с шагом сетки шаблона. Рецепты: карточки, строки, компактные строки, список, процесс, таймлайн, крупные цифры, диаграмма, таблица, сравнение, ключевая мысль (с опорными тезисами), картинка с текстом, листинг кода, план, разделитель, титул, финал. Кегли берутся только из шкалы шаблона, текст подгоняется по реальным метрикам шрифта (`textfit.py`), цвета проверяются на контраст WCAG (`style.py`). Листинг кода ставится на тёмную плашку моноширинным шрифтом стиля, пояснения — справа. Источник цифр и автор фото пишутся в подвал слайда вместо названия колоды: лицензии CC требуют указать автора.
- **Заполнение паттернов** (`patterns.py`) пишет текст в фигуры образца, снимает лишние пункты и равномерно раздвигает оставшиеся. Иконки образца заменяются тематическими (Lucide, `icons.py`) в цвете исходной, фото — иллюстрацией с обрезкой cover. Одинаковые роли соседних пунктов получают один кегль.
- **Сборщик** (`builder.py`) собирает PPTX: клоны слайдов, нативные таблицы и диаграммы python-pptx в цветах палитры, иконки, картинки, заметки спикера; затем удаляет исходные слайды шаблона. Диаграммы: столбцы, полосы, линия, круговая и кольцевая, накопительные (до 8 серий, подписи внутри сегментов цветом, читаемым на заливке), рассеяние (полупрозрачные точки и линия тренда второй серией); подписи осей и единицы, формат чисел, границы оси, подсветка категорий. У горизонтальных полос подпись каждой категории обязательна: кегль подгоняется под шаг строки, а пропуск подписей запрещается (`c:tickLblSkip = 1`) — иначе PowerPoint на невысокой диаграмме показывает подписи через одну.
- **Картинки** (`add_images` в оркестраторе) подбираются один раз на контент по `Brief.image_source` : `auto` — свободное фото Wikimedia Commons, если оно подошло, иначе иллюстрация генератора; `web` — только фото; `generate` — только генерация.
  - **Фото** (`research/photos.py`): кандидаты — поиск Commons по запросу иллюстратора и главные картинки прочитанных статей Википедии. Код отсеивает мелкие (меньше 1000 px), вертикальные, не JPEG/WebP, несвободные лицензии (NC, ND), схемы, карты, логотипы, сканы и снимки политиков; Qwen-vision (скилл `photo_picker`) по контактному листу выбирает одно фото или отказывается. Разные слайды не получают одно и то же фото. Автор и лицензия идут в подвал слайда, ссылка на страницу файла — в заметки.
  - **Иллюстрации** (`images.py`) генерируются GigaChat → Kandinsky в стиле шаблона. Стиль берётся из палитры словами: описание «векторная инфографика, иконки, интерфейс» тянет генератор в схемы с псевдобуквами. Промпт очищается от «3D», чисел, кавычек, названий брендов и слов про схемы и текст — модель рисует их буквально. Результаты кэшируются по хэшу промпта; генерация

идёт по одной картинке, при ответе 429 запрос повторяется с нарастающей паузой. Соединение с GigaChat устанавливается без проверки TLS-сертификата (только для этого хоста): сертификаты Минцифры могут отсутствовать в системе.

- Слайд-иллюстрация, которому не нашлось ни фото, ни рисунка, становится обычным списком.

Дизайнерский вариант B: модель компонует слайд

`backend/app/layout/freeform.py`, `ff_examples.py`. Включается в `config.yaml → freeform` (`enabled`, `variants: [B]`, `critic`, `attempts`).

Что рисует модель. Содержательные слайды типов «список», «карточки», «процесс», «таймлайн», «цифры», «сравнение», «главная мысль» на контентных холстах. Титул, план, разделители, финал, диаграммы, таблицы, слайды с картинкой (модель не видит фото), клоны образцов и слайды с блоком «и что?» остаются за рецептами.

Дизайн-система для модели (`Design`). Цвета по ролям (текст, заголовки, приглушённый, основной, акцент, вторичный, подложка, линия, фон), шкала кеглей шаблона по ролям (подпись, текст, вводная, заголовок, число, крупное), шрифты, хром слайда (кикер, заголовок, линия, подвал уже стоят) и контентная область в пикселях слайда 960×540. Модель (скилл `slide_designer`) отвечает примитивами: текст, плашка (прямоугольник, скруглённый, эллипс, шеврон, «домик», ромб, шестиугольник), линия или стрелка, иконка.

Чтобы соседние слайды не были одинаковыми, у каждого типа есть ротация композиций (например, для процесса — цепочка шевронов, блоки в два ряда со стрелками, нумерованные строки; для карточек — «бенто», крупные номера над колонками, сетка плашек, колонки с разделителями, нумерованные строки), и композиция двух предыдущих слайдов не повторяется. Модель получает подсказку композиции, образец её геометрии под число пунктов и область контента (сетка 8 px), опорную раскладку рецепта (размеры текста в ней уже проверены по метрикам шрифта) и что стоит на соседних слайдах.

Код проверяет каждую попытку. Ответ привязывается к сетке, удерживается в области, кегль подбирается по реальным метрикам шрифта; `repair` и `salvage` раздвигают наложившиеся плашки и ряды, убирают иконки, оторванные от своего текста; `diagnose` находит пустоты, мелкий текст, пропавшие тезисы. Попытка получает штраф: проблемы геометрии ×10, оторванные иконки и пропавшие заголовки пунктов ×5, разреженность +1. Следующая попытка идёт с конкретными замечаниями кода и прошлым вариантом после выравнивания, а не с «попробуй ещё». Остаётся лучшая попытка; если ни одна не стала чистой (штраф ≥ 10), слайд остаётся рецептом (предупреждение `freeform_fallback`, в событии — «рецептом — N»).

Попытки и дедлайн. В обычном режиме — до 3 попыток на слайд (`freeform.attempts`), и к 200-й секунде прогона (`pipeline.designer_s`) новые попытки не начинаются, а начатые обрываются: слайд остаётся лучшей чистой попыткой или рецептом. Так три колоды укладываются в 5 минут даже на медленном инференсе. В режиме Extended thinking — 4 попытки (`pipeline.extended.designer_attempts`) без дедлайна. Слайды варианта рисуются параллельно.

Критик по скриншоту. После сборки и рендера скилл `slide_critic` (vision) смотрит на картинку каждого нарисованного слайда вместе со списком элементов и диагностикой кода и возвращает исправленный список. Исправление принимается, только если оно не хуже: код его чинит без ошибок, наложений не больше, чем было, и текстов не меньше половины. Изменённые слайды пересобираются. В обычном режиме критик запускается, если до дедлайна прогона больше минуты; в Extended thinking — всегда.

8. Слой 4. Аудит

`backend/app/audit/`. Подробный каталог — в AUDIT.md; здесь общее устройство.

Два вида проверок. Всего 37 проверок: 26 детерминированных и 11 контекстных.

- **Детерминированные** (`deterministic.py`) работают по самому файлу PPTX и пикселям рендера LibreOffice. Размеры текста считаются по настоящим метрикам шрифтов шаблона, контраст — по фону под текстом на рендере, а не по заявленной заливке. Один и тот же слайд всегда даёт один и тот же результат.
- **Контекстные** (11 вопросов «да/нет»; `contextual.py`) задают VLM вопросы по картинке слайда вместе с его текстом и брифом (скилл `slide_auditor`), а LLM по тексту всей колоды (скилл `deck_auditor`) отвечает про единый язык и связность соседних слайдов. Слайды проверяются параллельно, у каждого вызова — потолок по дедлайну прогона. Для длинного материала аудитор получает не первые 3500 знаков, а начало брифа и фрагменты, где встречаются слова слайда.

Уровни замечаний в интерфейсе: «Предупреждение» (оранжевый), «Замечание» (синий), «Совет» (серый). Аудит подсказывает, что доработать, а не сообщает о поломке, поэтому слово «ошибка» и красный цвет не используются (`frontend/src/lib/format.ts`, `components/Audit.tsx`).

Ложные срабатывания фильтруются: «опечатка» засчитывается, только если слово есть в тексте слайда и правка небольшая; для титула, плана, разделов и финала часть вопросов не применяется; короткие заголовки-темы — формат шаблона, а не ошибка.

Исправления (`fixes.py`). Пользователь отмечает исправимые замечания, и `apply_fixes` правит план или IR:

- детерминированные: привести кегль к шкале, вернуть цвет из палитры, заменить шрифт, вернуть элемент в область, перекадрировать картинку, разбить перегруженный слайд, убрать лишние серии диаграммы, удалить дубль;
- через модель: заголовок-вывод, опечатки, неподтверждённые факты, сокращение текста.

После этого вариант пересобирается и проверяется снова (`rebuild_variant`). Детерминированный аудит повторяется целиком, контекстный — только по изменённым слайдам; номер ревизии растёт. В режиме Extended thinking исправимые предупреждения и замечания правятся автоматически, кругами с повторной проверкой (см. раздел 10).

9. Слой 5. Экспорт

- **PPTX** — нативные объекты на макетах шаблона; результат сборщика.
- **PDF** — рендер LibreOffice того же PPTX.
- **HTML** (`export/html.py`) — один файл: каждая страница PDF превращается в векторный SVG (`get_svg_image`, текст как контуры) во встроенном просмотрщике. Навигация клавишами `←` `→`, `F` — показ на весь экран, `Esc` — выход. К каждому слайду добавлен скрытый текст для экранных дикторов.
- **PNG-превью** — страницы PDF; интерфейс показывает их и отдаёт миниатюры WebP по параметру `?w=`.

10. Оркестрация: прогон, режимы, сроки, отказы

`backend/app/pipeline.py`. Слои вызываются строго по порядку и обмениваются только моделями из раздела 3.

Этапы прогона и события

Прогресс	Этап (`stage`)	Что происходит
2%	`style`	без шаблона компании: подбор встроенного стиля (или выбранный пользователем) и сборка его пака
2–3%	`brand`	брендбук: параметры стиля или производный шаблон компании; предупреждения — в событии
3%	`plan`	длинный материал читается целиком и сводится в выжимку
3–3,5%	`data`	анализ датасетов: природа, расчёты, визуализация (в событие кладётся карточка каждого датасета, включая нераспознанные с причиной)
3,5–4%	`research`	поиск фактов в Википедии (в событие кладутся источники)
4%	`plan`	планировщик пишет сценарий
10%	`data`	колода по данным: сюжет, главная мысль, выводы
18–20%	`plan`, `data`	сверка чисел с брифом, слайды данных, источники на слайдах; сценарий готов (в событие кладётся список слайдов)
20%	`outline`	только в режиме плана: прогон останавливается и ждёт `POST /api/jobs/{id}/build`
22–25%	`images`	фото Wikimedia Commons, затем иллюстрации генератора; один раз на контент, общие для всех вариантов
30–36%	`compose`	планы вариантов, композиция, сокращение не влезших текстов; при сокращении варианты собираются заново
42%	`build`	сборка PPTX и рендер PDF/PNG вариантов параллельно; у варианта B — дизайнер и критик (свои события)
70%	`audit`	детерминированный и контекстный аудит вариантов параллельно
80–85%	`audit`	только Extended thinking: круги автоисправления
93%	`export`	HTML
100%	`done` / `error`	итог

Каждое событие пишется в `job.json` и уходит подписчикам SSE (`/api/jobs/{id}/events`). Клиент, подключившийся позже, получает накопленные события и затем живой поток. Поток закрывается на `done`, `error` или `outline`.

Режимы генерации (`Brief.quality`)

	Обычный (`normal`)	**Extended thinking (`extended`)**
Общий срок	`pipeline.deadline_s` = 300 с: три колоды с аудитом не дольше 5 минут	`pipeline.extended.deadline_s` = 3600 с, фактически без лимита
Поиск фактов	не дольше `pipeline.research_s` = 40 с от начала прогона	без лимита
Дизайнер варианта B	до 3 попыток на слайд, заканчивает к `pipeline.designer_s` = 200 с	4 попытки (`designer_attempts`), без дедлайна
Критик по скриншоту	если до дедлайна больше 60 с	всегда
Контекстный аудит	заканчивается за 25 с до дедлайна	до общего срока
Автоисправление	нет: исправления выбирает пользователь	`fix_rounds` = 2 круга: исправимые предупреждения и замечания правятся (`apply_fixes`), вариант пересобирается и проверяется снова (`rebuild_variant`); круг не запускается, если исправлять нечего

Переключатель режима — в форме новой презентации (`[data-testid=quality]`).

Параллелизм

- Варианты собираются и рендерятся параллельно (`asyncio.to_thread`, число одновременных рендеров ограничено `render.max_parallel`); слайды варианта B дизайнер рисует параллельно.
- Контекстный аудит идёт параллельно по слайдам, внутри клиента моделей вызовы ограничены семафором `llm.max_concurrency`.
- Конспекты кусков длинного материала и чтение статей Википедии идут параллельно; датасеты считаются в отдельном потоке и не блокируют API.
- Фото подбираются по очереди (разные слайды не должны получить одно и то же фото), иллюстрации генерируются последовательно (ограничение тарифа GigaChat).

Сроки

Общий дедлайн обычного режима `pipeline.deadline_s` (по умолчанию 300 с, требование ТЗ). У каждого этапа с моделью свой потолок: поиск фактов — 40 с от начала, дизайнер варианта B — до 200-й секунды, критик — только если до дедлайна больше минуты. Контекстный аудит получает срок на 25 секунд раньше (дедлайн минус 25 с): слайды, до которых очередь не дошла, пропускаются, а колоды не теряются и остаются с детерминированным отчётом. Этап, не успевший к своему сроку, отдаёт то, что успел, или пропускается — прогон его не ждёт (`_bounded`).

Отмена

`POST /api/jobs/{id}/cancel` отменяет идущую задачу (`asyncio.Task` из `JOB_TASKS`), проект получает статус ошибки с сообщением «Отменено пользователем» и после этого его можно удалить: удаление идущей генерации запрещено (409).

Отказы и деградация

Что упало	Что происходит
Основная модель не ответила	две попытки на основной модели, затем две на запасной (только для текстовых вызовов и только если запасная задана и отличается от основной); ошибка JSON лечится отдельным запросом на починку
Планировщик вернул слишком мало слайдов	добор скиллом `deck_extender`; если он упал — колода без добора
Не удалось дописать заметки, тонкие слайды, иллюстрации или сократить текст	пропускается только этот шаг, колода собирается
Не прочитался кусок длинного материала	выжимка собирается из остальных кусков
Поиск фактов упал или не успел	колода строится по брифу, событие «ничего полезного не нашлось»
Не разобрался файл с данными	файл не попадает в колоду, причина видна в карточке и в событии `data`
Модель не ответила на этапе данных	заголовки, выводы и заметки — тексты кода; порядок датасетов — исходный
Дизайнер варианта B не дал чистого слайда или вышло время	слайд остаётся рецептом
Критик предложил худший вариант	исправление не принимается
Нет ни фото, ни ключа GigaChat, или ошибка генерации	слайды-иллюстрации превращаются в обычные, колода собирается без картинок
Шрифт брендбука не установлен, логотип шаблона не найден	применяется остальное, предупреждение в событии `brand`
VLM недоступна при разборе шаблона	профиль остаётся детерминированным, стиль помечается как «VLM недоступна»
Упал один из аудитов	остальные отчёты сохраняются, ошибка пишется в лог
Ошибка на любом этапе	задача переходит в `error`, сообщение сохраняется, событие уходит в поток
Перезапуск сервера посреди прогона	при чтении задача с диска в статусе «идёт» помечается ошибкой «Генерация прервана»; перезапуск — из интерфейса

11. Модели и скиллы

Подробнее о выборе моделей, инференсе и замеренных скоростях — в MODELS.md.

Клиент (`llm/client.py`)

- Асинхронный OpenAI-совместимый клиент; провайдер и модели задаются в `config.yaml` (по умолчанию Yandex Cloud AI Studio) и переопределяются переменными окружения `LLM_BASE_URL`, `LLM_TEXT_MODEL`, `LLM_VISION_MODEL`, `LLM_FALLBACK_MODEL`; для эндпоинтов, которые требуют заголовок `Project-ID`, — `LLM_PROJECT_ID`. Так подключается инференс VK.

- Семафор конкурентности, таймауты, повторы, переключение на запасную модель (если она задана и отличается от основной).
- Режим размышлений Qwen отключён через `chat_template_kwargs.enable_thinking=false`: для структурированного JSON он не нужен и только съедает время.
- Устойчивое извлечение JSON (ответ в блоке кода, текст до и после JSON, висячие запятые), при ошибке — один запрос на починку.
- Трасса всех вызовов в `llm_trace.jsonl`: скилл и версия, модель, попытка, время, токены, промпты и ответ; картинки в трассе заменены заглушкой.

Скиллы (`llm/skills.py`, `backend/skills/`)

Скилл — это файл `backend/skills/<имя>/<версия>.yaml` с описанием, видом (`text` или `vision`), температурой, лимитом токенов и парой промптов (`system`, `user`) с подстановкой `{{переменных}}`. Активная версия каждого скилла задаётся в `config.yaml → skills`. Промпты не правятся на месте: изменение — это новая версия файла. В журнал прогона пишутся имя, версия и хэш промпта.

Скилл	Активная версия	Вид	Роль
`template_analyst`	v1	vision	стиль, тон и стиль иллюстраций по контактному листу шаблона
`style_picker`	v2	text	три подходящих встроенных стиля по теме; берётся не повторяющий недавние колоды
`material_digest`	v1	text	конспект фрагмента длинного материала
`material_merge`	v1	text	сведение конспектов в одну выжимку
`clarifier`	v1	text	уточняющие вопросы перед генерацией
`web_researcher`	v1	text	поисковые запросы по теме
`wiki_picker`	v1	text	отбор статей Википедии из выдачи поиска
`wiki_reader`	v1	text	факты из статьи Википедии (числа — дословно)
`deck_planner`	v8	text	сценарий: структура, заголовки, тексты, данные, заметки, слайды с кодом
`deck_extender`	v1	text	добор слайдов до нужного объёма
`slide_writer`	v1	text	дописывает слайды после правки плана, сменившие тип и «тонкие» слайды
`speaker_notes`	v1	text	текст спикера к слайдам без заметок
`illustrator`	v2	text	промпты иллюстраций и запросы фото по теме колоды и тезисам слайда
`photo_picker`	v1	vision	выбор фото для слайда из найденных кандидатов
`data_analyst`	v2	text	выводы по датасетам и «и что?»: почему важно и что из этого следует
`data_story`	v1	text	сюжет колоды по данным: главная мысль, порядок, выводы, рекомендации

Скилл	Активная версия	Вид	Роль
`text_condenser`	v1	text	сокращение текста под лимит символов слота
`slide_designer`	v1	text	послайдовый дизайн: композиция слайда из примитивов дизайн-системы
`slide_critic`	v1	vision	самопроверка послайдового дизайна по скриншоту
`slide_auditor`	v1	vision	аудит слайда по картинке
`deck_auditor`	v2	text	ревью колоды: язык и связность
`fixer`	v1	text	правка содержимого слайда по исходным данным

История версий `deck_planner` (v1–v8) хранится в репозитории: v5 — жёсткие правила фактов, запрет подписей-заглушек, план совпадает с разделами, деликатные темы; v6 — длительность выступления, заметки к каждому слайду, строгий объём; v7 — данные брифа становятся диаграммой и таблицей, иллюстрации не занимают слайды с цифрами; v8 — слайды с кодом для учебных и технических колод. Список скиллов, версий и описаний отдаёт `GET /api/skills`.

12. Хранилище

Сервис не использует базу данных: всё лежит в каталоге данных (`DATA_DIR`, по умолчанию `data/`) в виде файлов.

```
data/
  templates/<sha256[:16]>/     template.pptx, profile.json, preview/, canvases/
  templates/style-<id>-<хэш>/  профиль пака встроенного стиля
  templates/<id>-brand-<bid>-b|t/   производный шаблон в стиле брендбука (derived.pptx, profile.json)
  brands/<bid>/                brand.json, логотипы и рендеры источников
  uploads/<sha256[:16]>__<имя> исходники файлов с данными
  jobs/<дата-время-id>/        job.json, content.json, llm_trace.jsonl,
                               analysis.json (датасеты), research.json (факты из сети), photos/,
                               A/ B/ C/: plan.json, layouts.json, audit.json,
                                         deck_<вариант>.pptx/.pdf, <вариант>.html, slides/
  cache/                       кэш картинок, шрифтов, паков стилей (styles/), профилей LibreOffice,
                               временных файлов, трасса уточняющих вопросов
```

Состояние активных задач держится в памяти процесса (`JobStore`) и дублируется в `job.json`; идущие задачи для отмены — в `JOB_TASKS`. Список проектов строится по файлам на диске. Идентификатор проекта — время создания и случайный хвост; идентификатор шаблона — начало sha256 файла, поэтому один и тот же шаблон не хранится дважды. Брендбук и датасет тоже адресуются хэшем содержимого.

13. HTTP API

`backend/app/api/main.py` (FastAPI). Полный перечень методов — в README; интерактивная документация — `/docs`.

Методы, появившиеся вместе с новыми возможностями:

Метод	Назначение
`GET /api/styles`	каталог встроенных стилей: палитра, шрифты, мотив, готов ли пак
`POST /api/styles/resolve`	стиль и свои цвета → `template_id` (первый раз собирает пак)
`GET /api/styles/{sid}/preview/{n}.png`	превью слайда пака стиля
`POST /api/brand`	брендбук (`multipart`, поле `files`: `.fig`, `.json`, `.pdf`, `.png` / `.svg` / `.jpg`, `.zip`): палитра, шрифты, логотипы, сетка, правила, предупреждения
`GET /api/brand/{id}` · `GET /api/brand/{id}/logo/{i}.png`	карточка брендбука и его логотипы
`POST /api/materials`	текст и датасеты из файлов (до 12): для таблиц — `dataset {id, tables[]}` с профилем, задачей и визуализацией или причиной «не распознано»
`POST /api/clarify`	уточняющие вопросы по брифу и объёму материала
`POST /api/jobs`	`template_id` необязателен (тогда нужен `brief.style_id`); `outline` — остановиться на плане; `variants` — собрать только эти варианты
`GET /api/jobs/{id}/outline` · `POST /api/jobs/{id}/build`	план колоды и сборка по правленому плану
`POST /api/jobs/{id}/cancel`	отмена идущей генерации
`POST /api/jobs/{id}/variants/{vid}/slides/{n}/chart`	другая визуализация слайда данных из предложенных: `{type}`

Особенности устройства:

- Разбор шаблона идёт в отдельном потоке; статус хранится в словаре `PARSE_STATUS` и отдаётся через `/api/templates/{id}/status`. Несколько шаблонов разбираются параллельно, каждый в своём потоке.
- Генерация запускается как асинхронная задача в цикле событий и не блокирует API; прогресс идёт через SSE с пингом каждые 15 секунд.
- `GET /api/jobs/{id}/variants/{vid}` у слайда данных отдаёт поле `data`: задача, выбранная визуализация, варианты, профиль, файл — по нему фронтенд строит панель «Визуализация».
- Миниатюры считаются лениво (WebP) и кэшируются рядом с исходным PNG; кэш пересоздаётся, если исходник новее.
- `POST /api/jobs/{id}/rerun` создаёт новый проект с тем же брифом на другом шаблоне и по умолчанию переиспользует `content.json`, поэтому сценарий не пересчитывается.
- `GET /api/templates` не показывает паки стилей и производные шаблоны брендбука.
- Тот же процесс раздаёт собранный фронтенд из `frontend/dist`.

14. Фронтенд

`frontend/` : React 19, Vite, Tailwind 4 с токенами VKUI (палитра VK), `@vkontakte/icons` , `@vkontakte/vk-bridge` , TanStack Query. В мини-приложении `vk-bridge` даёт тему ВК, скачивание файлов и «поделиться». Сборка использует относительные пути (`base: "./"`), поэтому работает и из FastAPI, и с хостинга мини-приложений.

Экраны (`src/pages/`):

Экран	Назначение
Главная (`Home`)	вход в сервис и сводная статистика (`/api/stats`)
Новая презентация (`NewProject`)	оформление → бриф → запуск; черновик брифа хранится в `localStorage`
Проекты (`Projects`)	список проектов
Проект (`Project`)	ход генерации, редактор плана, вкладки A/B/C, лента слайдов, просмотр, аудит с выбором исправлений, панель «Визуализация» у слайдов данных, сравнение вариантов, показ, экспорт, «Новая презентация в этом стиле»
Шаблоны (`Templates` , `TemplateDetail`)	библиотека шаблонов (загрузка нескольких разом) и разобранная дизайн-система

Новая презентация:

- **Оформление:** переключатель «Шаблон компании» (первый и по умолчанию) или «Стиль Лекала» — галерея встроенных стилей с мгновенным эскизом на клиенте, «Авто — подберу по теме» и своими цветами (`StyleGallery`). Ниже — необязательный блок брендбука (`BrandCard`): карточка с палитрой по ролям, шрифтами, логотипами, правилами и предупреждениями; для шаблона компании — выбор «важнее брендбук» или «важнее шаблон (только логотип)».
- **Материалы:** у файла с данными — карточка датасета: задача, предлагаемая визуализация, строки × колонки, ось времени, измерения, метрики, пропуски и поле «Цель (необязательно)»; нераспознанный файл помечен «Не распознано» с причиной.
- **Параметры:** режим генерации «Обычный» / «Extended thinking» (`[data-testid=quality]`); «Факты из интернета» (Википедия); источник картинок — «Фото, иначе рисунок», «Только фото», «Нарисовать».
- **Запуск:** нажатие на карточку A, B или C оставляет только этот вариант; галочка «Сначала показать план»; перед запуском при большом материале или коротком брифе — окно «Пара уточнений».

Общие компоненты: `Generation` (этапы и заготовки вариантов по SSE; необязательные этапы — брендбук, анализ данных, поиск фактов, иллюстрации; предупреждения брендбука и источники фактов), `OutlineEditor` (план: перетаскивание, тип, заголовок, тезисы, пожелание к слайду, добавление и удаление, «Собрать презентацию»), `VizPanel` (панель «Визуализация»), `Audit` (панель аудита: уровни «Предупреждение», «Замечание», «Совет»), `Viewer` (просмотр и показ), `SlideStack` , `CommandPalette` , `Cards` (карточки и загрузка шаблона: `Dropzone multiple` разбирает несколько файлов параллельно, у каждого своя строка с прогрессом), `StyleGallery` , `BrandCard` . API-клиент — `src/lib/api.ts` . Вёрстка адаптивная: на десктопе боковая панель, на телефоне нижняя навигация и нижние листы.

15. Как расширять систему

Задача	Что менять
Другая модель или провайдер	`config.yaml → llm` или переменные `LLM_BASE_URL`, `LLM_TEXT_MODEL`, `LLM_VISION_MODEL`, `LLM_FALLBACK_MODEL`, `LLM_PROJECT_ID`. Код не меняется, если API OpenAI-совместимый
Свой генератор изображений	интерфейс изолирован в `layout/images.py`: реализовать класс с методом `generate` и свойством `enabled`, подставить в `image_generator()`
Изменить промпт	новый файл `backend/skills/<имя>/vN.yaml`, затем `config.yaml → skills.<имя>: vN`. Старая версия остаётся для сравнения
Новый вариант вёрстки	запись в `config.yaml → variants`; для новой стратегии — колонка в `RECIPES` и правила в `variants.py`; послайдовый дизайн для варианта — `config.yaml → freeform.variants`
Новый рецепт вёрстки	метод `r_<имя>` в `Composer` (`layout/recipes.py`) и строка в `RECIPES`; использовать токены шаблона, `textfit` и `style.py`
Новая композиция дизайнера	строка в `HINTS` (`layout/freeform.py`) и генератор образца в `layout/ff_examples.py`
Новый встроенный стиль	файл `backend/app/styles/catalog/<id>.yaml` (`StyleSpec`); пак соберётся при первом использовании
Новый формат брендбука	функция `brand_from_<формат>` в `backend/app/brand/`, вид файла в `store.kind_of` и его ранг точности в `RANK`
Новая аналитическая задача или тип визуализации	расчёт в `analytics/analyze.py` (и `build_task`), баллы в `recommend.choose_tasks` / `recommend`, сборка в `build_chart`, тип диаграммы в `ChartSpec` и сборщике
Новый источник фактов	ветка в `research/web.py: research` и значение `config.yaml → search.provider`
Новая проверка аудита	функция в `audit/deterministic.py` (или вопрос в `contextual.py`), запись в `audit/catalog.py` (каталог виден интерфейсу и документации), обработчик в `audit/fixes.py`, тест
Порог проверки	`config.yaml → audit`
Сроки и режимы	`config.yaml → pipeline` (`deadline_s`, `research_s`, `designer_s`, `extended`)
Новый формат материалов	функция извлечения в `planning/materials.py` и расширение в `SUPPORTED`; для таблиц — `analytics/ingest.py`
Новый экспорт	функция рядом с `export/html.py`, вызов из `run_job` и `rebuild_variant`, формат в `download` API

Каждое изменение слоя должно сопровождаться тестом: модульные тесты не требуют моделей, интеграционные — только LibreOffice. Для новых частей есть отдельные наборы: `test_analytics.py`, `test_recommend.py`, `test_brand.py`, `test_styles.py`, `test_research.py`, `test_code.py`.

16. Принятые решения и компромиссы

- **Шаблон как правила, а не как подсказка модели.** Дизайн извлекается кодом и проверяется кодом. Цена — сложный разбор (шрифты, наследование стилей, группы), выигрыш — предсказуемость и работа на незнакомых шаблонах.
- **Клонирование образцов шаблона плюс композер.** Клоны дают «родной» вид, композер — покрытие случаев, для которых образца нет. Образец отбрасывается, если после заполнения остался бы чужой контент.
- **Варианты различаются вёрсткой, а не текстом.** Три сценария утроили бы стоимость и время и сделали бы варианты несопоставимыми. Одна контент-модель раскладывается тремя стратегиями.
- **Модель-дизайнер под контролем кода.** Вариант B даёт композиции, которых нет в рецептах, но каждая попытка проверяется геометрически, а критик по скриншоту не может сделать слайд хуже. Цена — время и вызовы модели, поэтому у дизайнера свой дедлайн и запасной путь — рецепт.
- **Встроенный стиль — тоже шаблон.** Стиль генерирует обычный `.pptx` и проходит тот же разбор, а брендбук превращает шаблон компании в производный шаблон. Так у пайплайна один вход, и все проверки работают одинаково.
- **Данные считает код, слова пишет модель.** Природа датасета, задача, тип диаграммы и все числа определяются правилами и воспроизводимы; модель пишет вывод и интерпретацию, а её неподтверждённые числа и причинные формулировки отбрасываются. Цена — шаблонные формулировки, когда модель не ответила.
- **Один мультимодальный Qwen для текста и зрения.** Разметка шаблона, подбор фото, критик и аудит по картинке идут тем же endpoint, что и тексты. Поэтому на инференсе VK основной — Qwen: модель, которая быстрее отвечает, но не принимает картинки, не закрыла бы vision-скиллы (см. MODELS.md).
- **Два режима вместо одного компромисса.** Обычный режим держит требование Т3 (три колоды за 5 минут) потолками времени на каждом этапе; Extended thinking снимает лимит ради лучшего результата: больше попыток дизайнера, критик всегда, автоисправление оформления.
- **Факты только из источников.** Сверка чисел до вёрстки и проверка `facts_supported` после исключают выдуманные цифры; источниками служат бриф, приложенные материалы, посчитанное по данным и, по желанию, Википедия с подписью источника. Цена — на бедном брифе без поиска сценарий получается качественным, без графиков.
- **Аудит внутри сервиса и с зоной замечания.** Пользователь видит, где проблема, и сам выбирает, что исправлять; автоматически правится только оформление и только в режиме Extended thinking.
- **Файлы вместо базы.** Каждый артефакт прогона доступен для проверки и воспроизведения; цена — сервис рассчитан на один процесс и не масштабируется горизонтально без внешнего хранилища.
- **Проверка в LibreOffice.** Он даёт превью и рендер для аудита на любой ОС; отрисовка в PowerPoint может немного отличаться (см. ограничения в README).
- **Иллюстрации через хостинговый сервис.** Это компромисс прототипа; способ перехода на полностью открытый контур описан в MODELS.md. Свободные фото Wikimedia Commons снижают зависимость от генератора.

Модели

Все модели — с открытыми весами, не больше 35B параметров, под лицензией Apache 2.0. Сервис не запускает свой инференс: модели вызываются через OpenAI-совместимый API. Провайдер задаётся в `config.yaml` (`llm.base_url`, `*_model`) и переменными окружения, поэтому переход между инференсом VK, Yandex Cloud AI Studio и собственным vLLM — это смена конфигурации, а не кода.

Роль	Модель	Параметры	Лицензия	Где используется
Текст (основная)	Qwen3.6-35B-A3B	35B всего, ~3B активных (MoE)	Apache 2.0	сценарий (`deck_planner`), добор слайдов до объёма (`deck_extender`), дописывание слайдов (`slide_writer`), текст спикера (`speaker_notes`), промпты иллюстраций (`illustrator`), выжимка длинных материалов (`material_digest`, `material_merge`), уточняющие вопросы (`clarifier`), подбор стиля (`style_picker`), поиск фактов (`web_researcher`, `wiki_picker`, `wiki_reader`), выводы по данным (`data_analyst`, `data_story`), послайдовый дизайн (`slide_designer`), сокращение текста (`text_condenser`), исправления (`fixer`), ревью колоды (`deck_auditor`)
VLM	Qwen3.6-35B-A3B (мультимодальная)	то же	Apache 2.0	описание стиля шаблона (`template_analyst`), выбор фото (`photo_picker`), критик дизайна по скриншоту (`slide_critic`), контекстный аудит слайдов по картинке (`slide_auditor`)
Текст (запасная)	gpt-oss-20b	21B (MoE)	Apache 2.0	автоматически, если основная модель не ответила или вернула невалидный JSON
Иллюстрации	Kandinsky через GigaChat API (Kandinsky 3.1, Kandinsky 3.0)	—	Apache 2.0 (веса Kandinsky 3.x)	генерация иллюстраций в стиле шаблона, если не нашлось свободного фото

Таблица описывает конфигурацию по умолчанию из `config.yaml` (Yandex Cloud AI Studio). Фото для слайдов модели не создают: это свободные снимки Wikimedia Commons, модель только выбирает подходящий.

Где работает инференс

Контур	Как подключается	Модели	Роль в сервисе
Инференс VK	переменные окружения `LLM_BASE_URL`, `LLM_TEXT_MODEL`, `LLM_VISION_MODEL` (и `LLM_FALLBACK_MODEL`); заголовок `Project-ID` — из `LLM_PROJECT_ID`	Qwen (`qwen38-27b`) — и текст, и зрение	основной инференс развёрнутого сервиса

Контур	Как подключается	Модели	Роль в сервисе
Yandex Cloud AI Studio	`config.yaml → llm` (`base_url`, `text_model`, `vision_model`, `fallback_model`), ключ `LLM_API_KEY` и каталог `LLM_FOLDER_ID`	Qwen3.6-35B-A3B — текст и зрение; gpt-oss-20b — запасная для текста	запасной контур: работает, когда `LLM_BASE_URL` и `LLM_*_MODEL` не заданы

Переменная окружения с именем модели перекрывает значение из `config.yaml`; пустое значение снимает модель (например, запасную). Запасная модель вызывается только для текстовых скиллов и только если она отличается от основной. Ключи и адреса эндпоинтов хранятся в `.env` и в репозиторий не попадают.

Замеренная скорость

Замеры команды: скорость генерации (токенов в секунду) и время до первого токена (TTFT):

Контур	Модель	Скорость, ток/с	TTFT	Зрение
Yandex Cloud AI Studio	qwen3.6-35b	≈236	0,5 с	есть
Yandex Cloud AI Studio	gpt-oss	≈203	—	нет
Инференс VK	qwen38-27b	≈80, в часы нагрузки — до ≈35	1,15 с	есть
Инференс VK	diona	≈106	—	нет: запрос с картинкой возвращает 502

На инференсе VK основной моделью выбран Qwen: diona быстрее, но не принимает картинки, а сервису нужно зрение — разметка шаблона, выбор фото, критик дизайна и аудит слайдов по картинке идут тем же endpoint, что и тексты.

Qwen на инференсе VK медленнее, и скорость падает в часы нагрузки. Поэтому в обычном режиме у каждого этапа с моделью свой потолок времени (`config.yaml → pipeline`: `deadline_s`, `research_s`, `designer_s`), и три колоды с аудитом укладываются в 5 минут даже на медленном инференсе: не успевший этап отдаёт то, что сделал, или пропускается. В режиме Extended thinking лимита нет (см. ARCHITECTURE.md).

Почему так

- **Qwen.** Одна модель закрывает и текст, и зрение: VLM-разметка шаблона, подбор фото, критик и аудит по картинке идут тем же endpoint. Qwen3.6-35B-A3B — MoE с ~3B активных параметров, отвечает за 1–2 с, поэтому десятки вызовов на три колоды укладываются в 5 минут. Режим размышлений отключён через `chat_template_kwargs.enable_thinking=false`: для структурированного JSON он не нужен и только съедает время.
- **gpt-oss-20b** — страховка от сбоев и таймаутов основной модели. Той же лицензии и размера хватает для сокращения текста и ревью.
- **Kandinsky.** В прототипе он вызывается через GigaChat API, это хостинговый сервис Сбера с собственными условиями использования. Для полностью открытого контура его заменяют развёрнутые у себя веса Kandinsky 3.x (Apache 2.0): интерфейс генератора изолирован в

`backend/app/layout/images.py`. Без ключа GigaChat колоды собираются со свободными фото Wikimedia Commons или без иллюстраций.

Как модели встроены

- **Клиент** (`backend/app/llm/client.py`):
  - асинхронный OpenAI-совместимый клиент; провайдер, модели и заголовок `Project-ID` задаются конфигурацией и переменными окружения;
  - семафор конкурентности (`llm.max_concurrency`; для Yandex Cloud — 4, потому что на каталог разрешено не больше 5 одновременных сессий);
  - таймауты, повторы и переключение на запасную модель;
  - извлечение и починка JSON из ответа;
  - трасса всех вызовов в `data/jobs/<id>/llm_trace.jsonl`: скилл, версия, хэш промпта, модель, время, токены.
- **Скиллы** (`backend/skills/<name>/<version>.yaml`): системный и пользовательский промпты с подстановкой `{{переменных}}`, вид (`text` или `vision`), температура, лимит токенов. Активные версии перечислены в `config.yaml → skills`, их список и описания отдаёт `GET /api/skills`.
- **Детерминизм прежде модели:**
  - все размеры, цвета, шрифты и геометрия вычисляются без моделей;
  - числа по таблицам с данными считает код, модель только формулирует вывод, и её числа сверяются с посчитанными;
  - модель-дизайнер варианта B отвечает примитивами, а привязку к сетке, область, кегль и наложения проверяет код;
  - модели пишут текст, описывают стиль и оценивают смысл.

  Поэтому сбой модели не ломает вёрстку. Разбор шаблона без VLM остаётся полным, слайд без ответа дизайнера собирается рецептом, а колода без контекстного аудита всё равно собирается и проверяется детерминированно.

Аудит

Аудит встроен в пайплайн. Каждый вариант после сборки проходит проверки, и отчёт показывается в интерфейсе рядом со слайдом: зона замечания подсвечивается на превью. Пользователь сам отмечает, что исправить. Исправления правят план или промежуточное представление, после чего вариант пересобирается и проверяется снова. Детерминированный аудит повторяется целиком, контекстный — только по изменённым слайдам. В режиме Extended thinking исправимые замечания оформления правятся автоматически (см. ниже).

Пороги задаются в `config.yaml → audit`: 6 буллетов, 15 слов в пункте, таблица 7×5, 5 серий, заполненность ¼–¾, контраст 4,5:1, не больше двух гарнитур.

Два вида проверок

- **Детерминированные** (`backend/app/audit/deterministic.py`) работают по самому файлу PPTX, который открывается python-pptx, и по пикселям рендера LibreOffice. Размеры текста считаются по настоящим метрикам шрифтов шаблона через FreeType, а не по числу символов. Контраст меряется по фону под текстом на рендере, а не по заявленной заливке, поэтому учитываются градиенты и картинки. Шрифты, кегли и цвета сверяются с токенами профиля шаблона.
- **Контекстные** (`backend/app/audit/contextual.py`) задают VLM вопросы по картинке каждого слайда вместе с его текстом и исходным брифом (скилл `slide_auditor`). По тексту всей колоды отвечает LLM (скилл `deck_auditor`): единый ли язык, связаны ли соседние слайды. Слайды проверяются параллельно, общий дедлайн прогона сохраняется: в обычном режиме контекстный аудит заканчивается за 25 секунд до дедлайна, вызов по слайду, не успевший к сроку, обрывается, и слайд остаётся с детерминированным отчётом. Для проверки фактов VLM получает не первые 3500 знаков источника, а начало брифа и фрагменты материала, в которых встречаются слова слайда: иначе факты из середины длинной методички аудитор считал выдуманными.

Уровни замечаний

У каждого замечания одна из трёх важностей. Аудит подсказывает, что доработать, а не сообщает о поломке, поэтому в отчёте аудита нет красного цвета и слова «ошибка».

Важность в отчёте (`severity`)	Подпись в интерфейсе	Цвет
`error`	Предупреждение	оранжевый
`warning`	Замечание	синий (акцент интерфейса)
`info`	Совет	серый

Подписи и цвета задаются в `frontend/src/lib/format.ts` (`SEVERITY`) и используются в панели аудита (`components/Audit.tsx`: кольцевая сводка и счётчики). Те же цвета — в карточках проектов (`components/Cards.tsx`: «без замечаний», «N предупр.», «N замеч.»), в точках на миниатюрах слайдов (`components/Viewer.tsx`) и во вкладках вариантов на странице проекта.

Каталог проверок

Всего 37: 26 детерминированных и 11 контекстных. Тот же список отдаёт API: `GET /api/audit/checks`.

Вёрстка

Код	Проверка	Тип	Как проверяется
`out_of_bounds`	Элемент вышел за границы слайда	детерм.	Габариты фигуры сравниваются с размером слайда (допуск 0,4%).
`overlap`	Два блока наложились друг на друга	детерм.	Пересечение прямоугольников > 3% меньшей площади, кроме вложения в контейнер-подложку.
`text_overflow`	Текст не поместился в свою рамку	детерм.	Высота текста считается по реальным метрикам шрифта (PIL/FreeType) и сравнивается с рамкой.
`text_cut`	Текст обрезан краем слайда	детерм.	Текстовая рамка выходит за пределы слайда.
`alignment`	Блоки не выровнены по направляющим	детерм.	Карточки ряда на одной высоте; левый край контента совпадает с направляющей макета.
`margins`	Контент заходит в поля у краёв	детерм.	Контент не выходит за контентную область холста (поля и зона колонтитула шаблона).
`image_stretched`	Картинка растянута, пропорции нарушены	детерм.	Соотношение сторон исходного изображения с учётом обрезки равно соотношению рамки (допуск 3%).

Шаблон

Код	Проверка	Тип	Как проверяется
`font_not_in_template`	Шрифт не из шаблона	детерм.	Гарнитура каждого абзаца есть в типографике шаблона; моноширинный шрифт стиля и стандартные моноширинные (Courier New, Consolas, JetBrains Mono) в блоках кода разрешены.
`font_families`	Гарнитур больше двух	детерм.	Не более двух гарнитур на слайде; моноширинный шрифт блока кода не в счёт.
`size_not_in_scale`	Кегль не из типографической шкалы	детерм.	Кегль входит в шкалу, извлечённую из шаблона (допуск 0,25 pt).
`color_not_in_palette`	Цвет не из палитры шаблона	детерм.	Цвет совпадает с токеном палитры или его оттенком (смесь с фоном/белым/чёрным).
`layout_not_from_template`	Слайд собран не на макете шаблона	детерм.	Макет слайда присутствует среди макетов исходного шаблона.
`chrome_moved`	Логотип или колонтитул сдвинут	детерм.	Элементы-«хром» шаблона стоят на исходных координатах.

Код	Проверка	Тип	Как проверяется
`contrast`	Контраст текста к фону ниже 4,5:1	детерм.	WCAG 2.1 по пикселям рендера под текстом; крупный текст (≥18 pt или ≥14 pt полужирный) — 3:1.

Плотность

Код	Проверка	Тип	Как проверяется
`too_many_bullets`	Больше 6 буллетов на слайде	детерм.	Считаются маркированные абзацы и заголовки пунктов.
`long_bullet`	Буллет длиннее 15 слов	детерм.	Подсчёт слов в каждом пункте.
`table_size`	Таблица больше 7 строк или 5 колонок	детерм.	Размер нативной таблицы.
`chart_series`	Больше 5 серий на диаграмме	детерм.	Число c:ser в части диаграммы.
`fill_ratio`	Слайд заполнен меньше ¼ или больше ¾	детерм.	Площадь объединения объектов относительно площади слайда.

Целостность

Код	Проверка	Тип	Как проверяется
`file_open`	Файл не открывается	детерм.	Файл открывается python-pptx, LibreOffice рендерит все страницы.
`placeholder_text`	Остался текст-заглушка	детерм.	Регулярные выражения: lorem ipsum, XXX, TODO, «вставьте текст», «Имя Фамилия», «Вывод 1»…
`empty_slide`	Пустой слайд или слайд с одним заголовком	детерм.	На контентном слайде нет объектов, кроме заголовка.
`raster_slide`	Слайд оказался картинкой	детерм.	Одна картинка ≥90% площади и ни одного текстового объекта.
`chart_labels`	У диаграммы нет подписей осей, единиц или легенды	детерм.	Разбор XML диаграммы: подписи значений, заголовок оси, легенда для нескольких серий. У плотных диаграмм (рассеяние, линия длиннее 14 точек, больше 20 категорий) подписи значений сняты нарочно — достаточно подписи оси.
`duplicate_slides`	Два слайда дублируют друг друга	детерм.	Мера Жаккара по словам текстов слайдов > 0,8.

Контент

Код	Проверка	Тип	Как проверяется
`language`	Колода не на одном языке	детерм.	Доля латиницы среди букв слайда > 30%.

Код	Проверка	Тип	Как проверяется
`title_conclusion`	Заголовок содержит вывод, а не тему	VLM/LLM	VLM по картинке слайда (вопрос 1).
`content_matches_title`	Содержимое соответствует заголовку	VLM/LLM	VLM (вопрос 2).
`one_sentence`	Слайд пересказывается одним предложением	VLM/LLM	VLM (вопрос 3).
`facts_supported`	Цифры и факты есть в исходных материалах	VLM/LLM	VLM со сверкой с брифом (вопрос 4).
`has_content`	На слайде есть содержание	VLM/LLM	VLM (вопрос 5).
`visuals_relevant`	Картинки и иконки относятся к теме	VLM/LLM	VLM (вопрос 6).
`no_garbage`	Нет служебного мусора	VLM/LLM	VLM (вопрос 7).
`typos`	Текст без опечаток	VLM/LLM	VLM (вопрос 8).
`deck_language`	Вся колода на одном языке	VLM/LLM	LLM по тексту колоды (вопрос 9).
`table_relevant`	Строки таблицы и легенда работают на мысль	VLM/LLM	VLM (вопрос 10).
`transition`	Соседние слайды связаны по логике	VLM/LLM	LLM по тексту колоды (вопрос 11).

Исправления

Действие	Для каких проверок	Как работает
`condense`	`text_overflow`, `long_bullet`	LLM сокращает текст под лимит символов конкретного слота, вариант пересобирается
`snap_size`	`size_not_in_scale`	Кегль приводится к ближайшему значению шкалы шаблона
`recolor`, `recolor_text`	`color_not_in_palette`, `contrast`	Ближайший цвет палитры; для текста — первый цвет палитры, дающий нужный контраст
`refont`	`font_not_in_template`, `font_families`	Замена на гарнитуру шаблона
`remove_shape`	`placeholder_text`	Удаление фигуры-заглушки
`clamp`	`out_of_bounds`, `text_cut`, `margins`	Элемент возвращается в контентную область холста
`recompose`	`overlap`, `alignment`, `empty_slide`, `chart_labels`, `chrome_moved`	Слайд пересобирается другим рецептом или холстом; хром шаблона берётся с холста на исходных местах
`recrop`	`image_stretched`	Картинка заново обрезается cover-кадрированием под рамку

Действие	Для каких проверок	Как работает
`enlarge`, `split`	`fill_ratio`, `too_many_bullets`, `table_size`	Укрупнение блоков на пустом слайде или разбиение перегруженного на два
`trim_series`	`chart_series`	Лишние серии диаграммы убираются (остаются 5 главных)
`drop_slide`	`duplicate_slides`	Дубль удаляется
`set_title`	`title_conclusion`, `content_matches_title`	Заголовок-вывод, предложенный VLM
`fix_typos`	`typos`	Замены «ошибка → исправление» из ответа VLM
`remove_unsupported`, `llm_fix`	`facts_supported`, `no_garbage`, `deck_language`	LLM (`fixer`) правит содержимое слайда по исходным данным

После исправления (и после смены визуализации слайда данных) вариант пересобирается общей функцией `rebuild_variant`: PPTX, PDF и PNG заново, детерминированный аудит целиком, контекстный — только по изменённым слайдам, номер ревизии растёт. Контекстные замечания неизменённых слайдов сохраняются, исправленные убираются из отчёта.

Автоисправление в режиме Extended thinking

В режиме Extended thinking (`Brief.quality = "extended"`) после первого аудита исправимые замечания с важностью «Предупреждение» и «Замечание» правятся автоматически — теми же действиями `apply_fixes`, что и по выбору пользователя. Все варианты исправляются параллельно, затем пересобираются и проверяются снова. Кругов не больше `config.yaml → pipeline.extended.fix_rounds` (по умолчанию 2); круг не запускается, если исправлять нечего. Советы и неисправимые замечания остаются пользователю. В обычном режиме автоисправления нет: три колоды должны уложиться в 5 минут, а что править — решает человек.

Защита от ложных срабатываний

Числа, которых нет в брифе, по возможности убираются ещё до вёрстки — это сверка с брифом, см. ARCHITECTURE. Аудит `facts_supported` остаётся второй линией: он ловит и нечисловые утверждения.

Контекстная модель иногда ошибается, поэтому её ответы фильтруются.

- **Опечатка засчитывается, только если «ошибочное» слово действительно есть в тексте слайда и отличается от исправления.** Иначе модель находит «опечатку» вида «поведение» → «поведение».
- **Стилистическая перефразировка не считается опечаткой.** Засчитывается только правка в пределах одного-двух слов, близкая по написанию («превет» → «привет»), а не «маму» → «свою мать».
- **Для титула, плана, раздела и финала** не применяются вопросы о заголовке-выводе, соответствии содержания и пересказе одним предложением.
- **Короткие заголовки-темы — это формат шаблона.** Если шаблон так пишет (2–3 слова в среднем), «нет вывода в заголовке» понижается до совета.
- **Однострочный текст, зашедший только на внутренние поля рамки**, не считается переполнением: так устроены декоративные цифры и заголовки многих шаблонов, и текст при этом не обрезается.
- **Слайды на паттернах шаблона** (текст в фигурах образца, таблица образца) не считаются пустыми.

- **Моноширинный шрифт блока кода** — третья гарнитура по делу: он не считается чужим шрифтом и не входит в лимит двух гарнитур.
- **Плотные диаграммы без подписей значений** (рассеяние, линия длиннее 14 точек, больше 20 категорий) не считаются диаграммами без подписей, если у них есть заголовок оси: подписи на них сняты нарочно.
- **Факты из длинного материала.** Аудитор фактов видит фрагменты источника, относящиеся к слайду, а не только начало материала, поэтому факт из середины документа не помечается как неподтверждённый.
- **Числа на слайдах данных** посчитаны кодом по приложенным таблицам и попадают в источник для сверки; неподтверждённые числа модели отбрасываются ещё до вёрстки (см. ARCHITECTURE).

Все фильтры покрыты тестами (`backend/tests/test_units.py`).